Báo cáo chi tiết triển khai Website Công thức Nấu ăn Recipe

Báo cáo mô tả quá trình xây dựng, triển khai và kiểm tra Đề 29 trên Ubuntu bằng Docker Compose. Nội dung đi từ yêu cầu và cấu hình nguồn đến thao tác sử dụng, kết quả kiểm thử và cách vận hành. Phần bìa được bổ sung riêng khi hoàn thiện hồ sơ nộp.

Nội dung báo cáo

Phần nội dung	Trang
1 Mục tiêu và tiêu chí thực hiện	2
2 Kiến trúc và luồng truy cập	3
3 Chuẩn bị môi trường Ubuntu	4
4 Quản lý mã nguồn trên GitHub	5
5 Quy trình khởi tạo bằng Docker Compose	7
6 Thiết kế cơ sở dữ liệu Recipe	10
7 Giao diện website và danh sách công thức	13
8 Cấu hình Nginx reverse proxy	22
9 Luồng giám sát bằng Prometheus và Grafana	24
10 Cấu hình Loki Promtail và nguồn log	32
11 Hardening được kiểm chứng tại runtime	37
12 Kiểm thử chức năng ứng dụng và database	39
13 Vận hành và các vấn đề đã xử lý	42
Phụ lục danh mục hình và nguồn minh chứng	44

Hình giao diện được chụp từ các dịch vụ đang chạy. Hình kết quả lệnh là bản định dạng lại output thực tế, có file TXT và JSON nguồn đi kèm để đối chiếu; đây không phải ảnh chụp cửa sổ Terminal. Các sơ đồ được xây dựng từ cấu hình triển khai.

Các thao tác tạo và sửa dữ liệu minh họa dùng bản ghi tạm riêng, sau đó được dọn đúng ID. Sáu công thức mẫu và các dữ liệu có sẵn khác được giữ nguyên.

1 Mục tiêu và tiêu chí thực hiện

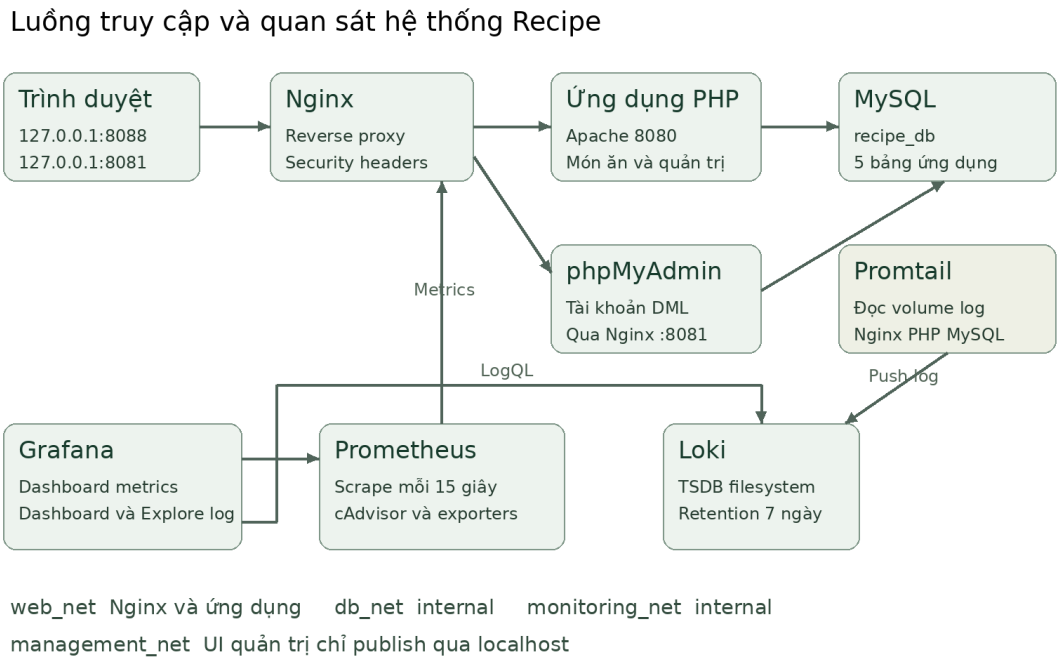
Đề 29 yêu cầu triển khai website chia sẻ công thức nấu ăn, có món ăn, nguyên liệu và danh mục; sử dụng MySQL và phpMyAdmin. Các thành phần vận hành gồm GitHub, Nginx reverse proxy, Prometheus, Grafana, Loki, Promtail và các biện pháp hardening. Báo cáo cung cấp đường dẫn source, thao tác tái lập và minh chứng cho từng nhóm yêu cầu.

Tiêu chí	Sản phẩm đã triển khai	Minh chứng chính
GitHub 1,5 điểm	Source, cấu hình, README và commit có ý nghĩa	Repository và lịch sử commit
Ứng dụng và DB 1,5 điểm	PHP Recipe, MySQL, phpMyAdmin	Giao diện, SQL, kiểm thử CRUD
Reverse proxy 1,5 điểm	Nginx chuyển tiếp website và DB tool	Cấu hình, nginx -t, header HTTP
Giám sát 1,5 điểm	Metrics container, web và database	Targets, PromQL, dashboard
Log tập trung 1,5 điểm	Ba nguồn log, Loki và Promtail	Bốn truy vấn LogQL có dữ liệu
Hardening 1,5 điểm	Non-root, mạng nội bộ, giảm quyền	Docker inspect, DB grants, HTTP
Tổng thể 1 điểm	Docker Compose, báo cáo và kịch bản demo	Kiểm thử, ảnh và hướng dẫn

Phương án chọn ứng dụng PHP tự xây theo cách tổ chức thư mục của repository mẫu Blog CMS. Mã Recipe và dữ liệu công thức được xây dựng cho đề tài này. WordPress đã cài thử trước đó không còn là ứng dụng phục vụ chính; các bảng và volume cũ được giữ để bảo toàn dữ liệu.

Kết quả tích hợp mới nhất ghi nhận 51/51 kiểm tra đạt lúc 00:00:41 ngày 24/10/2026. Kiểm tra cài mới độc lập từ GitHub đã đạt 8/8 ở commit 9393eaa. Hai mốc kiểm tra được phân biệt trong phần kiểm thử.

2 Kiến trúc và luồng truy cập



Hình 1 Kiến trúc ứng dụng và các thành phần quan sát

Nginx là điểm nhận HTTP của website tại 127.0.0.1:8088. Request được chuyển tới Apache PHP tại app:8080. PHP dùng PDO để đọc hoặc cập nhật MySQL qua hostname db. Cổng 8081 của Nginx chuyển tới phpMyAdmin; database không mở cổng trực tiếp ra host.

Lớp	Thành phần	Vai trò
Giao diện	PHP và CSS SVG	Danh sách, tìm kiếm, chi tiết và quản trị
Dữ liệu	MySQL và phpMyAdmin	Lưu công thức, kiểm tra và quản lý dữ liệu
Giám sát	Prometheus Grafana exporters	Theo dõi trạng thái và tài nguyên
Nhật ký	Promtail Loki Grafana	Gom log và tìm sự kiện

Dữ liệu nghiệp vụ nằm trong MySQL. Metrics nằm trong Prometheus; log tập trung nằm trong Loki. Grafana truy vấn hai kho quan sát này để hiển thị. Việc tách vai trò giúp tìm được cả thời điểm bất thường từ biểu đồ và các sự kiện liên quan từ log.

3 Chuẩn bị môi trường Ubuntu

1. Mở Terminal tại thư mục dự án trên Ubuntu. Kiểm tra Docker Engine, Compose, Git, GitHub CLI và Python 3.

2. Xác nhận Docker daemon hoạt động và tài khoản hiện tại có thể chạy docker compose.

3. Kiểm tra các cổng dự kiến sử dụng để tránh trùng với ứng dụng khác trên máy.

```
cd -- "/home/longphan/DTC245220017-De 29"
docker --version
docker compose version
git --version
gh --version
python3 --version
```

Phiên bản công cụ trên Ubuntu

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
Linux 7.0.0-38-generic
Ubuntu 24.04.2 LTS
Docker version 29.7.2, build a7dcaa6
Docker Compose version v5.5.0
git version 2.43.0
gh version 2.45.0 (2025-07-18 Ubuntu 2.45.0-1ubuntu0.3)
Python 3.12.3
```

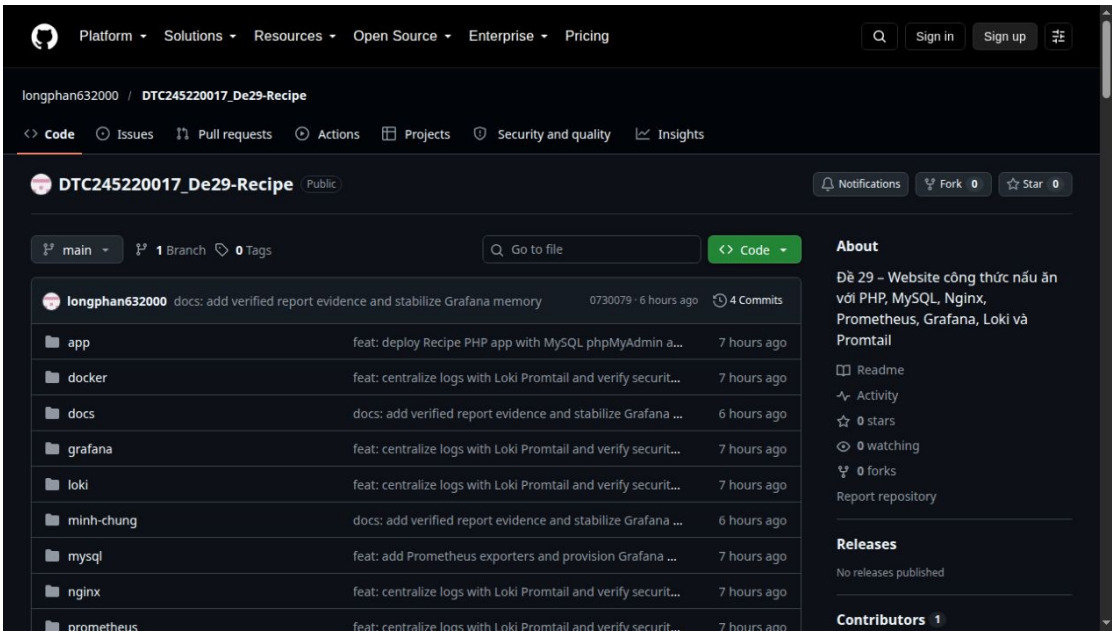
Hình 2 Phiên bản công cụ được kiểm tra trực tiếp trên Ubuntu

Cổng trên localhost	Chức năng
8088	Website và trang quản trị Recipe
8081	phpMyAdmin qua Nginx
3000	Grafana
9090	Prometheus
3100	Loki API

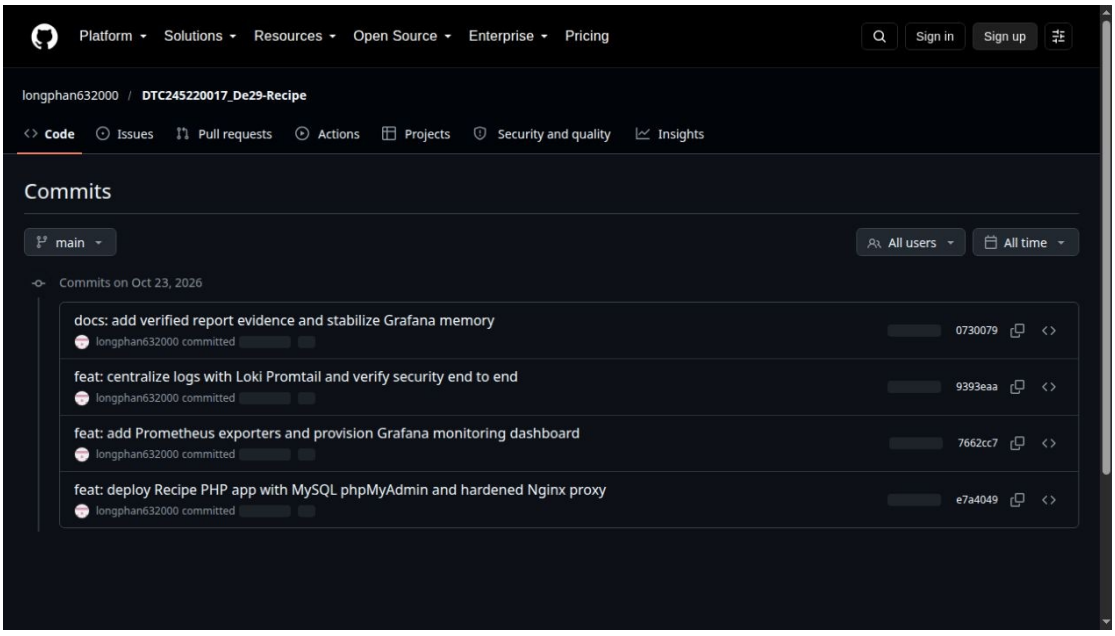
Các URL localhost phải được mở trong Ubuntu đang chạy Docker. Khi truy cập từ máy khác, dùng SSH tunnel tới Ubuntu. Giữ các UI quản trị trên loopback giúp giảm phạm vi truy cập trong môi trường lab.

4 Quản lý mã nguồn trên GitHub

Repository của sinh viên là https://github.com/longphan632000/DTC245220017_De29-Recipe. Tên repo chứa mã DTC245220017 và chủ đề Recipe. Repository công khai chứa mã nguồn, cấu hình và tài liệu hướng dẫn; các bí mật chạy thật được loại khỏi Git.



Hình 3 Repository GitHub có source và các thư mục cấu hình



Hình 4 Lịch sử bốn mốc commit tại thời điểm chụp minh chứng

Ba commit kỹ thuật đầu tương ứng triển khai ứng dụng và Nginx, giám sát, log tập trung và hardening. Các commit tài liệu sau bổ sung báo cáo và ảnh minh chứng.

Cấu trúc source và ý nghĩa các commit

Đường dẫn	Trách nhiệm
app/src	PHP, template HTML, CSS và sáu hình SVG
app/Dockerfile	Build PHP Apache chạy UID 33 và cổng 8080
mysql/init.sql	Tạo schema và dữ liệu công thức ban đầu
docker/	Dockerfile và cấu hình hỗ trợ MySQL phpMyAdmin Nginx Loki Promtail
nginx/nginx.conf	Reverse proxy, headers, log JSON và giới hạn request
prometheus/	Danh sách target và luật cảnh báo
grafana/	Provisioning datasource và dashboard JSON
loki/ và promtail/	Lưu log tập trung và cấu hình nguồn file
scripts/	Khởi tạo, kiểm thử, sao lưu và thu minh chứng
docs/ và minh-chung/	Báo cáo, hướng dẫn, ảnh, kết quả API và TXT

Các mốc commit của repository

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
e7a4049 23/10/2026 16:42
feat: deploy Recipe PHP app with MySQL phpMyAdmin and hardened Nginx proxy
7662cc7 23/10/2026 16:46
feat: add Prometheus exporters and provision Grafana monitoring dashboard
9393eaa 23/10/2026 17:03
feat: centralize logs with Loki Promtail and verify security end to end
0730079 23/10/2026 17:30
docs: add verified report evidence and stabilize Grafana memory
```

Hình 5 Các commit kỹ thuật và thời điểm từ git log

Cách chia thư mục tham khảo repository Khanhslee/DTC245220024_De7-Blog-CMS. Việc tham khảo giới hạn ở khung tổ chức hệ thống; nội dung Recipe, provisioning và bộ kiểm thử được xây cho Đề 29.

Các lệnh quản lý repository

Lần đầu tạo repo dùng git init, commit rồi GitHub CLI. Những lần cập nhật sau chỉ stage đúng thay đổi, commit và push; không tạo lại remote đã tồn tại.

```
git init -b main
git add .
git commit -m "feat: deploy Recipe PHP app with MySQL and Nginx"
gh repo create longphan632000/DTC245220017_De29-Recipe \
  --public --source=. --remote=origin --push
git push origin main
```

5 Quy trình khởi tạo bằng Docker Compose

1. Clone repository và chuyển vào thư mục source.

2. Chạy setup.py để tạo bí mật cục bộ, chuẩn bị secret phpMyAdmin và kiểm tra Compose.

3. Script build image, chạy MySQL đến khi healthy rồi nạp schema và dữ liệu mẫu.

4. Script giới hạn quyền DB, tạo tài khoản exporter, chạy các dịch vụ còn lại và khởi tạo quản trị viên bằng password hash.

5. Kiểm tra trạng thái container, HTTP và chức năng bằng verify.py.

```
git clone https://github.com/longphan632000/DTC245220017_De29-Recipe.git
cd DTC245220017_De29-Recipe
python3 scripts/setup.py
docker compose ps -a
python3 scripts/verify.py
```

Vì sao cần script setup

Ứng dụng chạy bằng tài khoản DB chỉ có quyền DML nên không tự tạo bảng khi khởi động. Setup thực hiện thao tác quản trị cần thiết trong giai đoạn khởi tạo rồi cấp SELECT, INSERT, UPDATE, DELETE cho tài khoản ứng dụng. Trình tự này tách quyền triển khai schema khỏi quyền chạy website.

Nếu .env chưa tồn tại, script tạo mật khẩu ngẫu nhiên riêng cho database, ứng dụng và Grafana. Nếu đã có .env, script giữ giá trị hiện tại. .env đặt quyền 0600, thư mục secrets đặt 0700; các file này không được đưa lên GitHub.

Bước trong script	Điều kiện hoặc kết quả
docker compose config --quiet	YAML và phép nội suy biến hợp lệ
docker compose build	Tạo các image tùy chỉnh theo Dockerfile
up --wait db	Database vượt healthcheck trước khi nạp dữ liệu
Nạp mysql/init.sql	Schema idempotent và dữ liệu tiếng Việt utf8mb4
Cấp quyền và khởi tạo admin	Mật khẩu băm, quyền DB theo vai trò
up --wait toàn bộ	Khởi chạy hệ thống bằng một project Compose

Các thuộc tính Compose của ứng dụng

```
app:
  user: "33:33"
  read_only: true
  networks: [web_net, db_net]
  depends_on:
    db:
      condition: service_healthy
  mem_limit: 384m
```

Đây là phần trích của service app. Dockerfile cài pdo_mysql, chuyển Apache sang cổng 8080, COPY source với chủ sở hữu UID 33 và khai báo USER 33:33. tmpfs cung cấp nơi ghi file tạm cho Apache khi root filesystem chỉ đọc.

Container và volume sau khi triển khai

Trạng thái dịch vụ Docker Compose

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
SERVICE          STATE     <no value>   PORTS
app              running   healthy      80/tcp, 8080/tcp
cadvisor         running   healthy      8080/tcp
db               running   healthy      3306/tcp, 33060/tcp
grafana          running                127.0.0.1:3000->3000/tcp
loki             running                127.0.0.1:3100->3100/tcp
mysql-exporter   running                9104/tcp
nginx            running   healthy      80/tcp, 127.0.0.1:8081->8081/tcp,
127.0.0.1:8088->8080/tcp
nginx-exporter   running
phpmyadmin       running                80/tcp, 8080/tcp
prometheus       running                127.0.0.1:9090->9090/tcp
promtail         running
```

Hình 6 Mười một dịch vụ của project Recipe đang chạy

Named volume của project Recipe

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
VOLUME NAME              DRIVER
recipe_app_logs          local
recipe_grafana_data      local
recipe_loki_data         local
recipe_mysql_data        local
recipe_mysql_logs        local
recipe_nginx_logs        local
recipe_prometheus_data   local
recipe_promtail_data     local
recipe_wordpress_data    local
```

Hình 7 Các named volume được Docker quản lý

App, MySQL và Nginx có healthcheck kiểm tra điều kiện phục vụ riêng. Một số dịch vụ hiển thị running mà không có Docker healthcheck; mức hoạt động của chúng được kiểm chứng thêm bằng HTTP, datasource health hoặc metrics target trong bộ kiểm thử.

Named volume giữ database, dữ liệu Grafana, metrics, log và vị trí đọc Promtail qua các lần tạo lại container. Dừng bằng docker compose stop không xóa dữ liệu. Lệnh down -v sẽ xóa volume và không dùng khi cần giữ kết quả.

Phân tách mạng và cổng truy cập

Mạng và thành viên đang chạy

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
recipe_db_net  internal=True
  recipe-app-1, recipe-db-1, recipe-mysql-exporter-1, recipe-phpmyadmin-1
recipe_management_net  internal=False
  recipe-grafana-1, recipe-loki-1, recipe-prometheus-1
recipe_monitoring_net  internal=True
  recipe-cadvisor-1, recipe-grafana-1, recipe-loki-1, recipe-mysql-exporter-1,
recipe-nginx-1, recipe-nginx-exporter-1, recipe-prometheus-1, recipe-promtail-1
recipe_web_net  internal=False
  recipe-app-1, recipe-nginx-1, recipe-phpmyadmin-1
```

Hình 8 Các mạng Docker và thành viên tại thời điểm kiểm tra

Cổng loopback và bảo vệ file bí mật

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

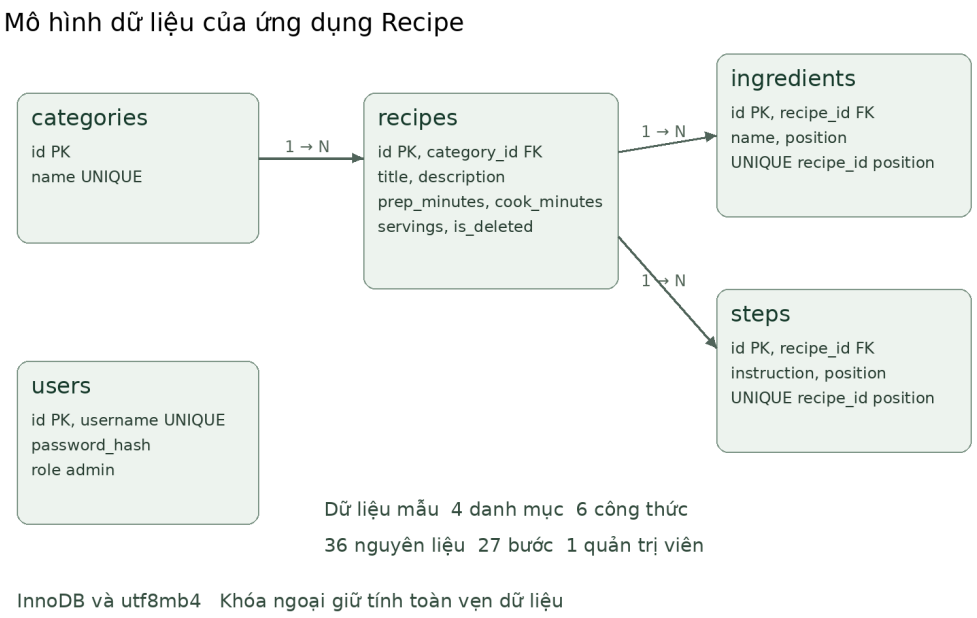
```
grafana 127.0.0.1:3000 -> 3000/tcp
loki 127.0.0.1:3100 -> 3100/tcp
nginx 127.0.0.1:8088 -> 8080/tcp
nginx 127.0.0.1:8081 -> 8081/tcp
prometheus 127.0.0.1:9090 -> 9090/tcp
db: không publish cổng
app, phpmyadmin: chỉ truy cập qua mạng Docker
.env mode 0o600
secrets mode 0o700
git check-ignore:
.env
secrets/phpmyadmin-secret.php
```

Hình 9 Cổng chỉ bind localhost và quyền file bí mật

db_net và monitoring_net là mạng internal. web_net nối Nginx với app và phpMyAdmin. management_net nối các UI giám sát với cơ chế publish cổng trên host. Thành viên của mỗi mạng được giới hạn theo nhu cầu kết nối; app không được nối trực tiếp với monitoring_net.

Network internal giúp giới hạn đường đi mạng nhưng không thay thế xác thực dịch vụ. Các cổng publish đều dùng HostIp 127.0.0.1; MySQL và các exporter không có cổng host.

6 Thiết kế cơ sở dữ liệu Recipe



Hình 10 Quan hệ giữa năm bảng dữ liệu của ứng dụng Recipe

Bảng	Dữ liệu chính	Quan hệ
categories	Tên danh mục	Một danh mục có nhiều món
recipes	Tên, mô tả, thời gian, khẩu phần, độ khó	category_id tham chiếu categories
ingredients	Tên nguyên liệu và thứ tự	recipe_id tham chiếu recipes
steps	Hướng dẫn và thứ tự bước	recipe_id tham chiếu recipes
users	Tên đăng nhập, hash mật khẩu, vai trò	Tài khoản quản trị nội dung

Schema dùng InnoDB và utf8mb4. Khóa ngoại giữ liên kết giữa món ăn và các dòng nguyên liệu, cách làm. Ràng buộc thứ tự theo từng công thức giúp kết quả được trình bày ổn định. is_deleted phục vụ lưu trữ và khôi phục thay vì xóa ngay công thức.

Trích khai báo bảng nguyên liệu

```
CREATE TABLE IF NOT EXISTS ingredients (
 id INT AUTO_INCREMENT PRIMARY KEY, recipe_id INT NOT NULL,
 name VARCHAR(500) NOT NULL, position SMALLINT UNSIGNED NOT NULL,
 UNIQUE KEY uq_ingredient(recipe_id,position),
 FOREIGN KEY(recipe_id) REFERENCES recipes(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Khởi tạo dữ liệu và kiểm tra phpMyAdmin

1. Trong setup, nạp mysql/init.sql sau khi MySQL healthy.

2. Mở http://127.0.0.1:8081, đăng nhập bằng MYSQL_USER và MYSQL_PASSWORD trong .env.

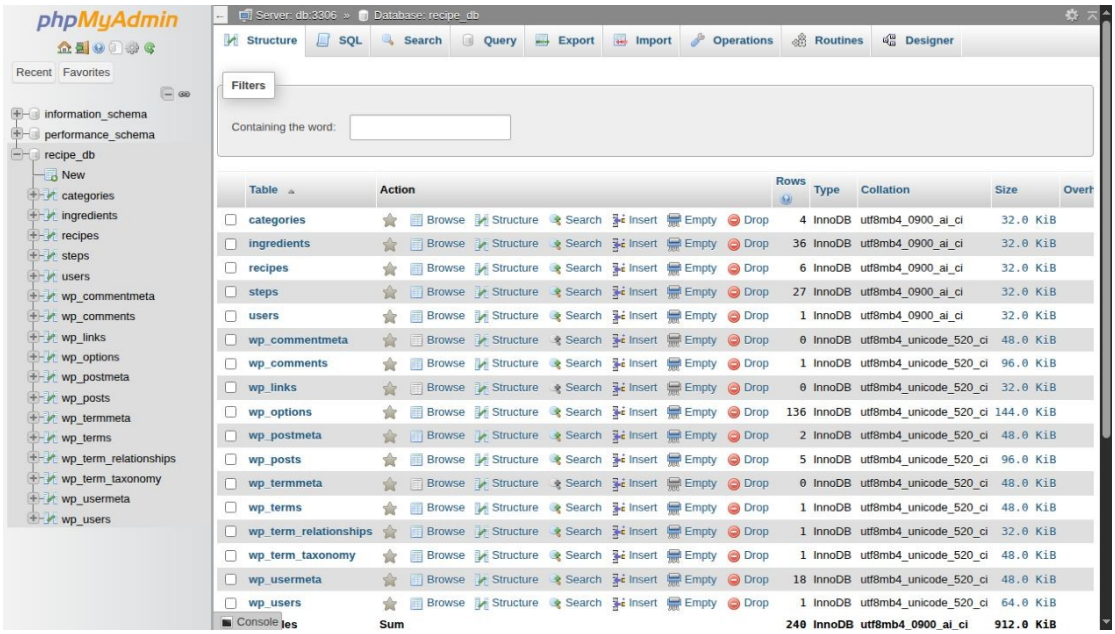
3. Chọn recipe_db, đối chiếu số dòng và engine của các bảng ứng dụng.

Số dòng dữ liệu và quyền tài khoản ứng dụng

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
bang    so_dong
categories      4
recipes 6
ingredients     36
steps   27
users   1
Grants for recipe_user@%
GRANT USAGE ON *.* TO `recipe_user`@`%`
GRANT SELECT, INSERT, UPDATE, DELETE ON `recipe_db`.* TO `recipe_user`@`%`
```

Hình 11 Số dòng dữ liệu và quyền DML thực tế của recipe_user



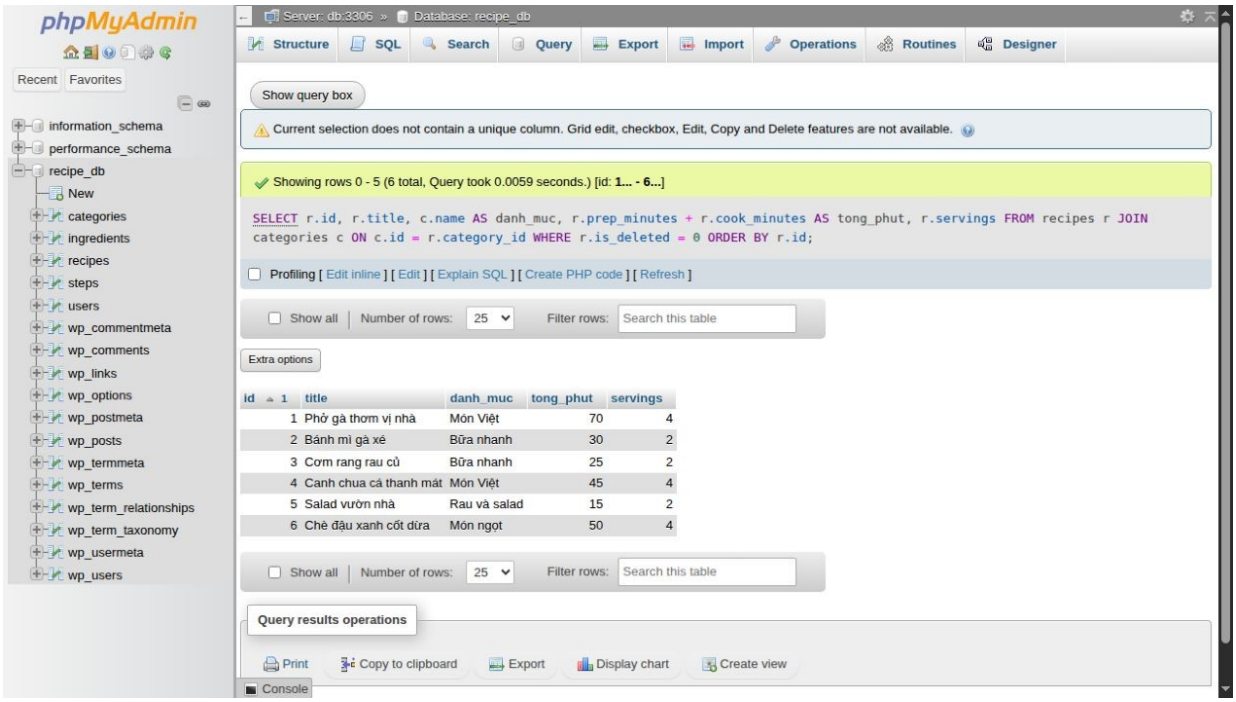
Hình 12 Năm bảng Recipe và các bảng WordPress cũ được giữ trên máy

Dữ liệu mẫu có 4 danh mục, 6 món, 36 nguyên liệu, 27 bước và 1 quản trị viên. Các bảng wp_ là dữ liệu từ lần cài thử trước, không được ứng dụng PHP sử dụng. Cài mới từ GitHub chỉ tạo đúng năm bảng ứng dụng.

Đối chiếu dữ liệu bằng truy vấn SQL

Tại phpMyAdmin, chọn recipe_db rồi mở tab SQL. Câu truy vấn sau nối món ăn với danh mục, tính tổng thời gian và chỉ lấy công thức công khai.

```
SELECT r.id, r.title, c.name AS danh_muc,
       r.prep_minutes + r.cook_minutes AS tong_phut,
       r.servings
FROM recipes r
JOIN categories c ON c.id = r.category_id
WHERE r.is_deleted = 0
ORDER BY r.id;
```



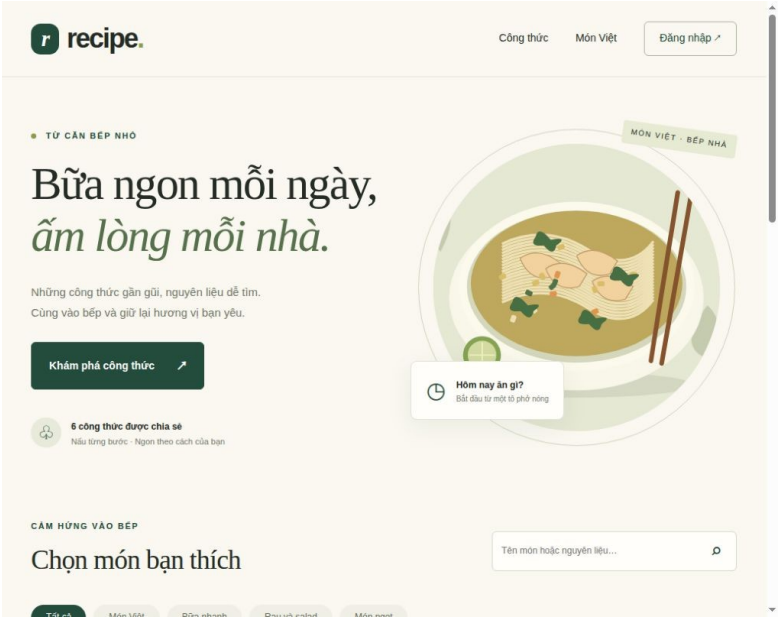
Hình 13 Truy vấn JOIN trả sáu công thức cùng danh mục thời gian khẩu phần

Kết quả phở gà 70 phút và 4 người được đối chiếu với trang chi tiết của website. Cách kiểm tra này xác nhận giao diện lấy dữ liệu từ database, đồng thời minh họa quan hệ recipes và categories.

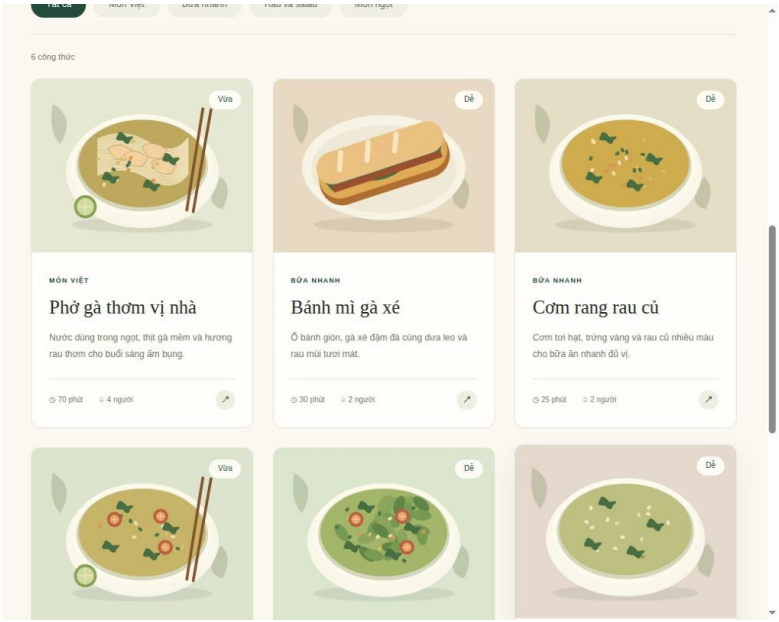
Thông báo phpMyAdmin về việc không có cột duy nhất để sửa trực tiếp lưới kết quả JOIN không phải lỗi thực thi SELECT. Truy vấn vẫn trả dữ liệu; chỉnh sửa nghiệp vụ thực hiện qua trang quản trị ứng dụng.

7 Giao diện website và danh sách công thức

Người dùng mở http://127.0.0.1:8088 qua Nginx. Trang chủ giới thiệu website; phần danh sách hiển thị thẻ món ăn, danh mục, thời gian, khẩu phần và độ khó. Các hình SVG nằm trong source nên giao diện không phụ thuộc dịch vụ ảnh bên ngoài.



Hình 14 Trang chủ Recipe được truy cập qua cổng Nginx

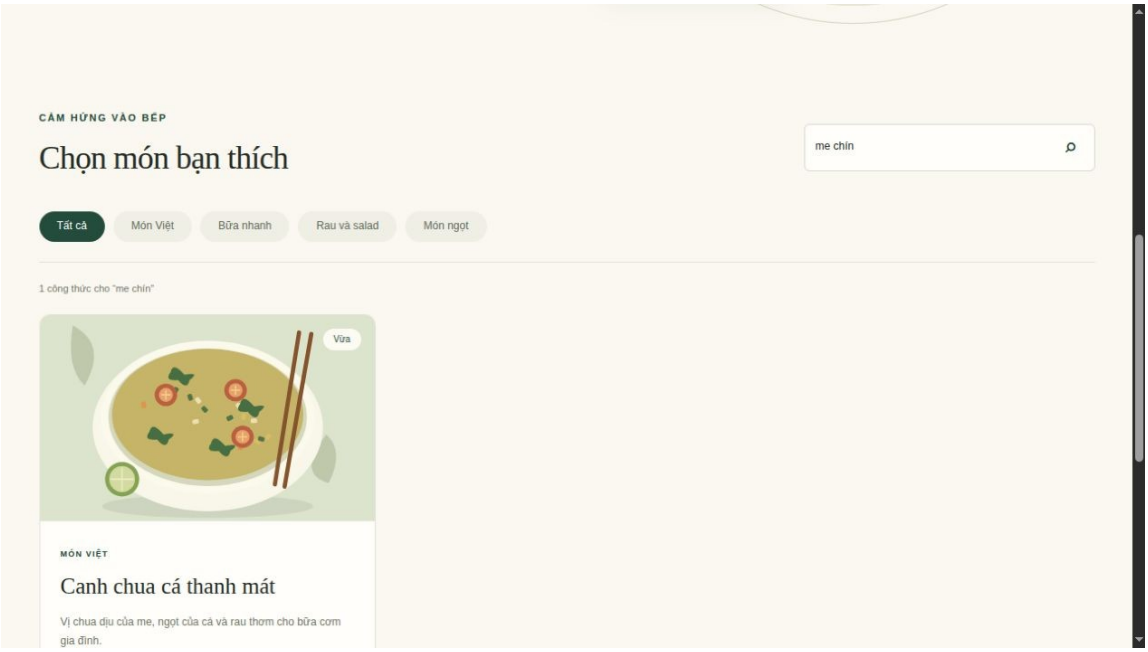


Hình 15 Các thẻ công thức và bộ lọc danh mục

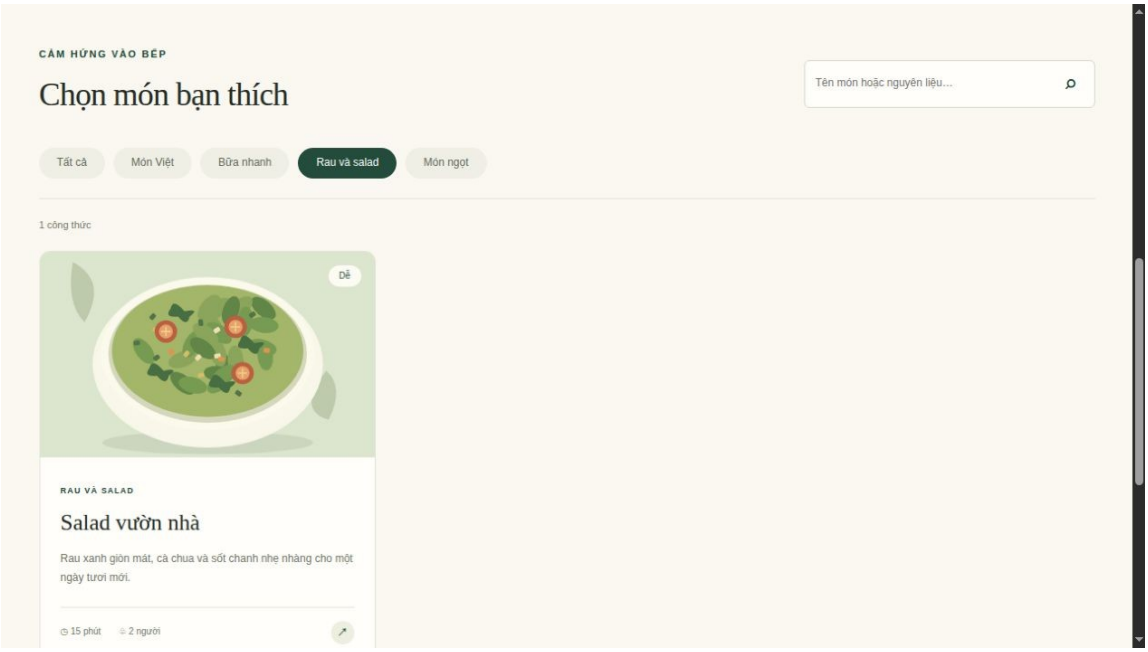
Số công thức lấy bằng COUNT trong MySQL. Nội dung danh sách được truy vấn theo điều kiện is_deleted=0 để loại món đã lưu trữ.

Thao tác tìm nguyên liệu và lọc danh mục

Nhập me chín vào ô tìm kiếm rồi chạy tìm. Ứng dụng tìm đồng thời trong tên món, mô tả và bảng nguyên liệu; kết quả là một công thức canh chua. Chọn Rau và salad để kiểm tra bộ lọc danh mục.



Hình 16 Từ khóa me chín trả đúng món canh chua

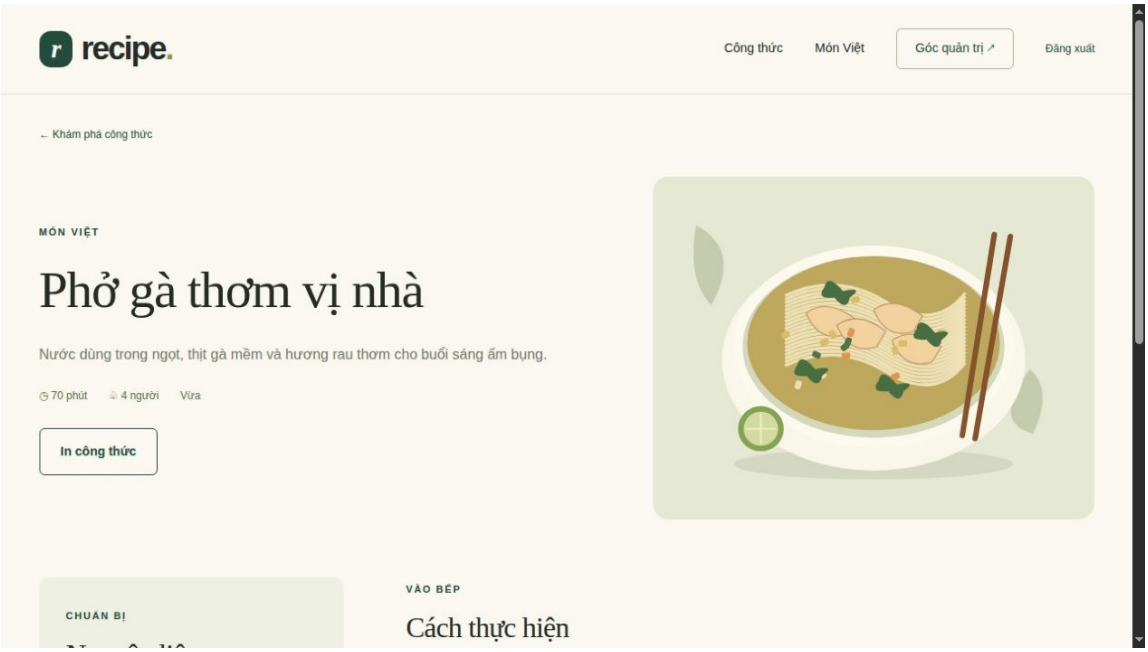


Hình 17 Danh mục Rau và salad trả một công thức salad

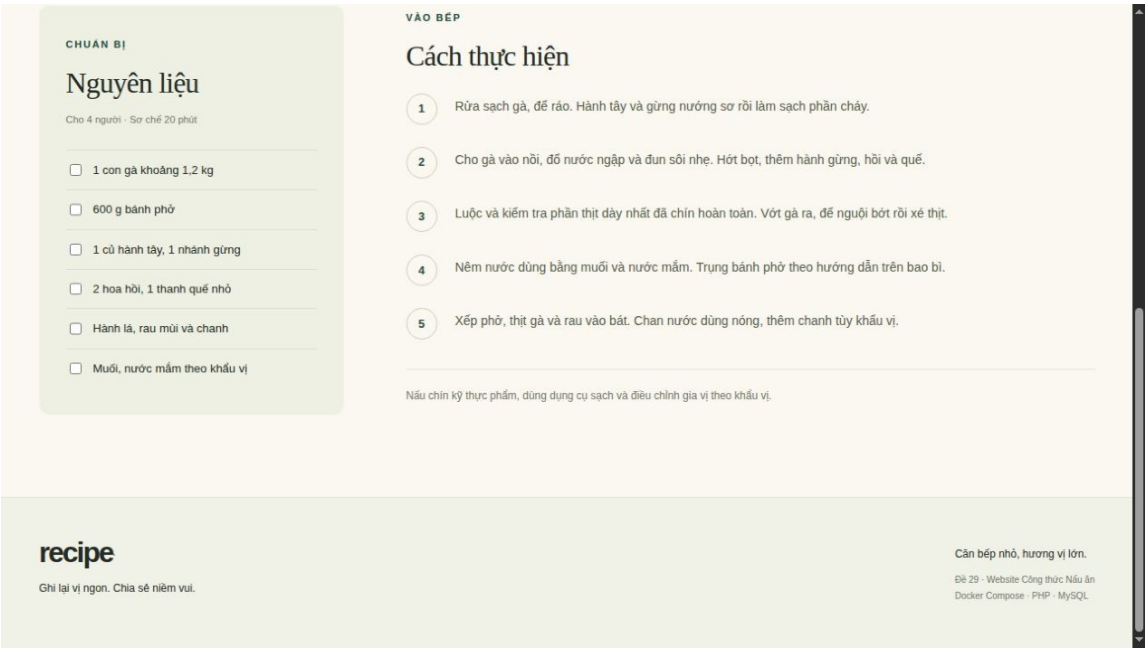
Câu SQL dùng tham số PDO và điều kiện EXISTS trên ingredients. Bộ lọc danh mục thêm điều kiện category_id; từ khóa được escape khi đưa lại lên HTML.

Thao tác xem chi tiết và chuẩn bị nguyên liệu

Mở Phở gà thơm vị nhà từ danh sách. Kiểm tra tổng thời gian 70 phút, khẩu phần 4 người, sáu nguyên liệu và năm bước. Các dòng nguyên liệu có checkbox để đánh dấu trong lúc chuẩn bị.



Hình 18 Thông tin món ăn thời gian và khẩu phần từ database



Hình 19 Nguyên liệu và hướng dẫn được sắp theo thứ tự

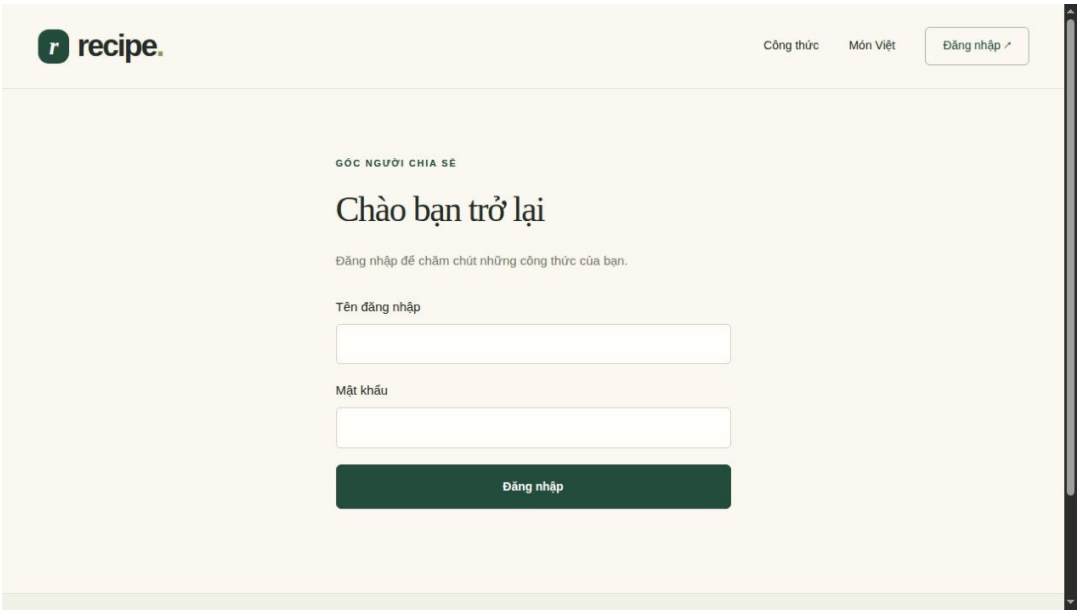
Nút In công thức dùng CSS dành cho bản in. Checkbox chỉ hỗ trợ thao tác trên giao diện, không cập nhật database.

Đăng nhập và mở trang quản trị

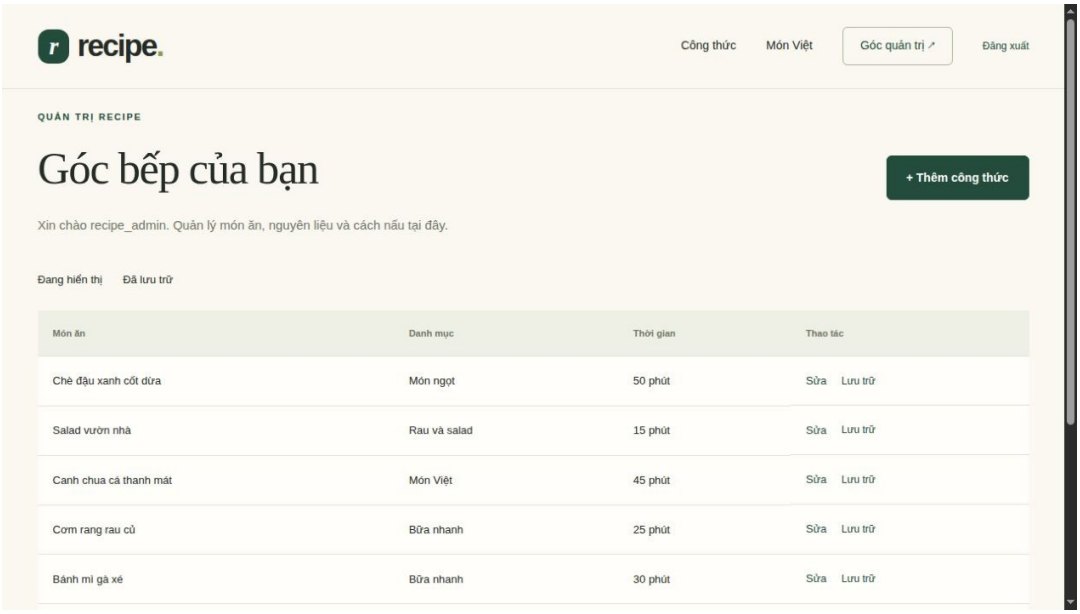
1. Mở đường dẫn ?page=login. Nhập APP_ADMIN_USER và APP_ADMIN_PASSWORD từ .env.

2. Khi đăng nhập thành công, hệ thống đổi session ID và chuyển đến trang quản trị.

3. Trang quản trị cung cấp thêm món, sửa món, danh sách đang hiển thị và danh sách lưu trữ.



Hình 20 Biểu mẫu đăng nhập quản trị trước khi nhập mật khẩu

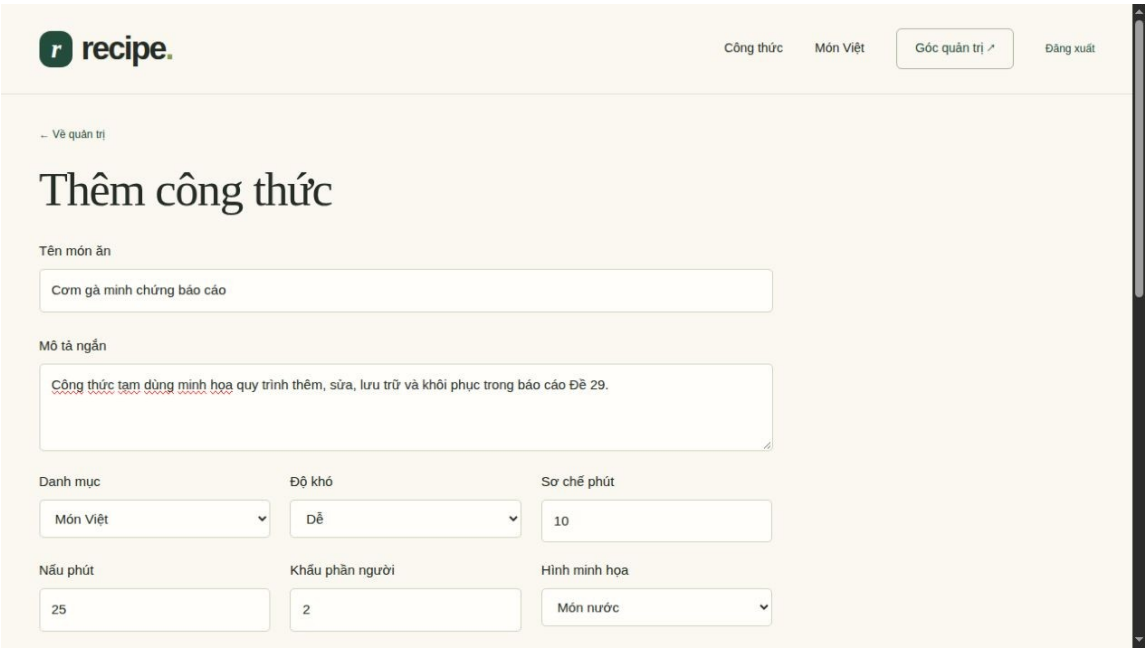


Hình 21 Trang quản trị hiển thị danh sách công thức hiện có

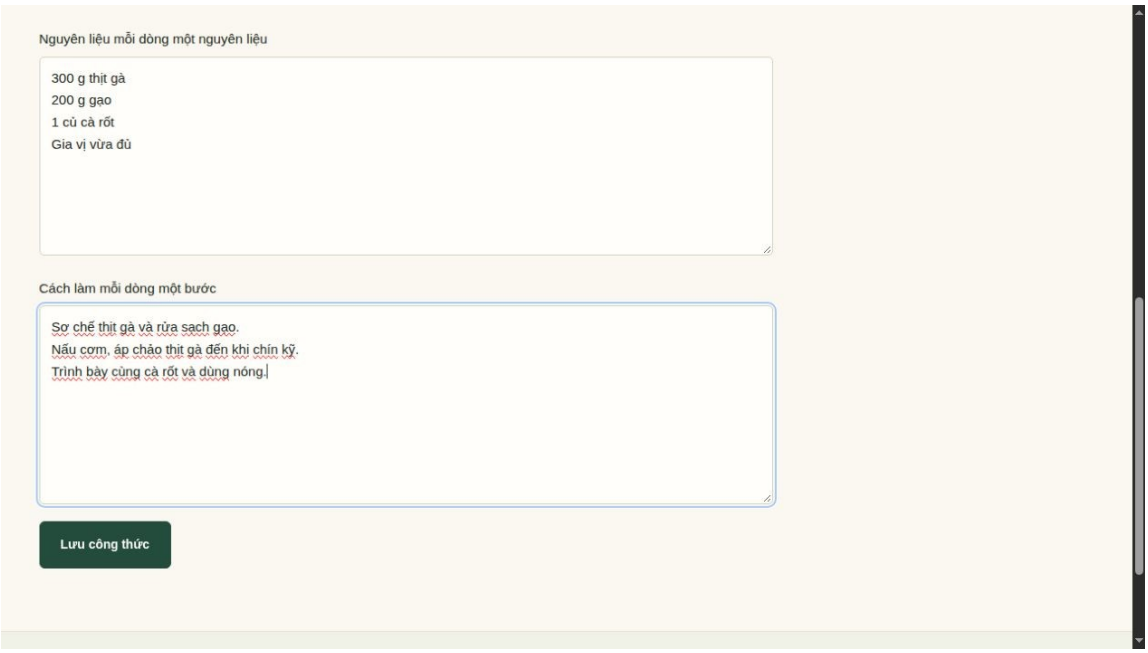
Người chưa có session hợp lệ được chuyển về đăng nhập khi mở ?page=admin hoặc ?page=edit. Tài khoản quản trị được tạo bằng hash mật khẩu trong bảng users.

Thao tác thêm công thức mới

Từ trang quản trị chọn Thêm công thức. Bản ghi minh họa dùng tên Cơm gà minh chứng báo cáo, sơ chế 10 phút, nấu 25 phút, 2 người. Nguyên liệu và cách làm được nhập mỗi dòng một mục.



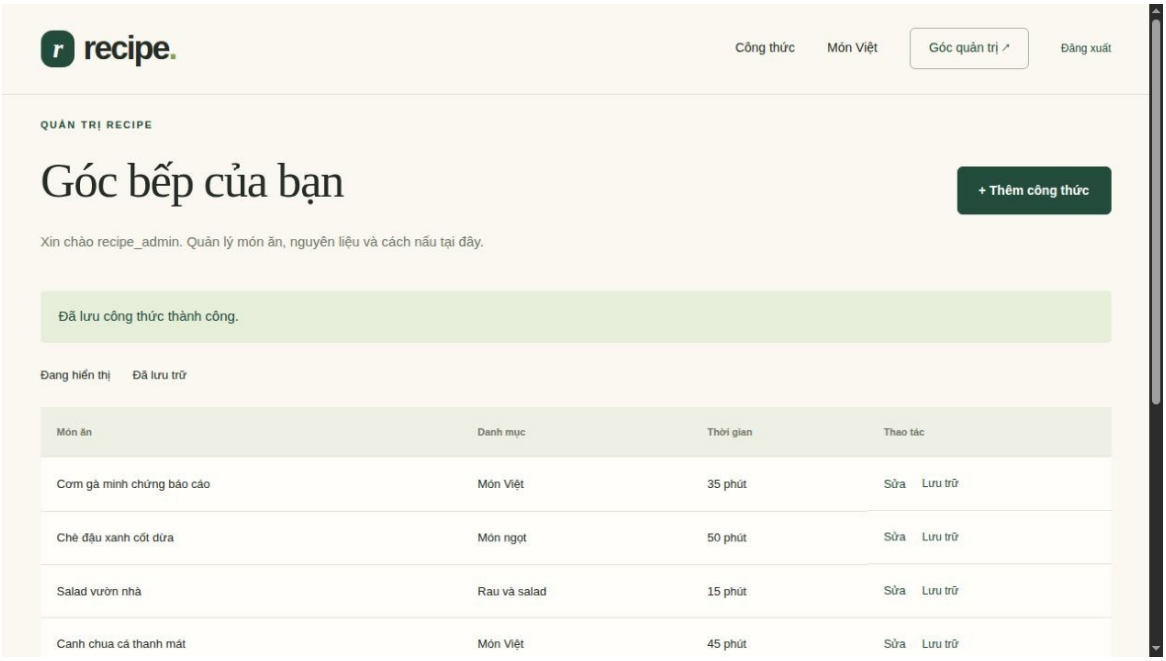
Hình 22 Nhập tên mô tả danh mục thời gian và khẩu phần



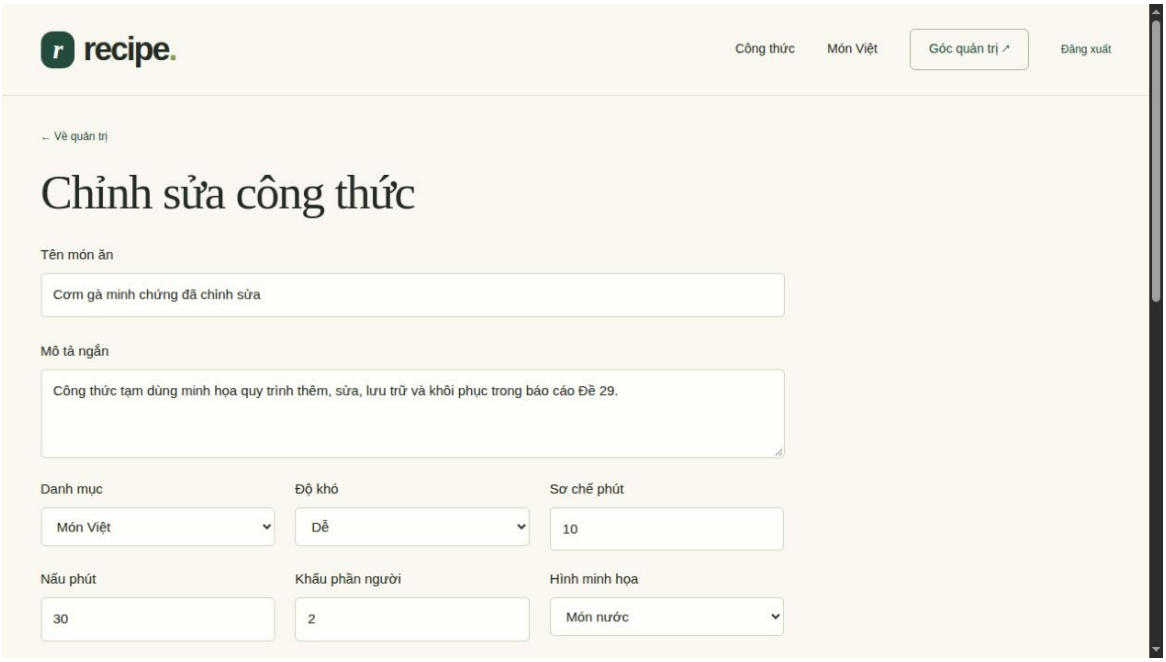
Hình 23 Nhập nguyên liệu cách làm và nút lưu công thức

Form kiểm tra trường bắt buộc, danh mục hợp lệ, số người từ 1 đến 50 và mỗi thời gian không quá 600 phút. Lưu dữ liệu món và các dòng con trong cùng transaction.

Xác nhận tạo mới và mở biểu mẫu chỉnh sửa



Hình 24 Thông báo lưu thành công và món mới có tổng thời gian 35 phút

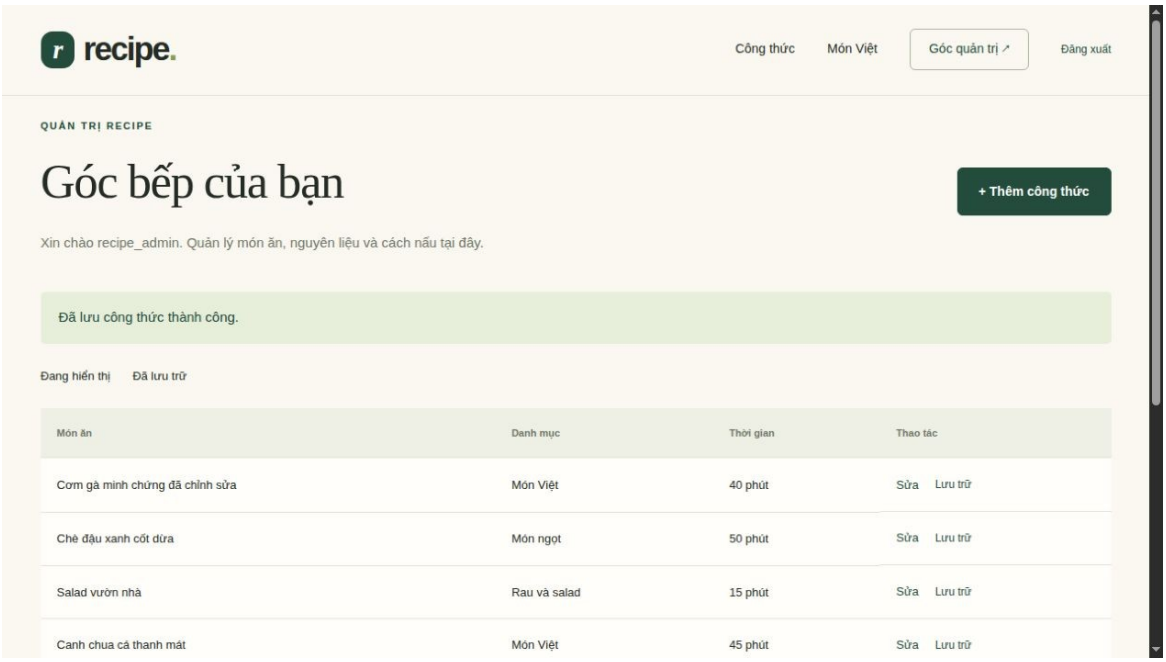


Hình 25 Biểu mẫu chỉnh sửa cùng bản ghi minh họa

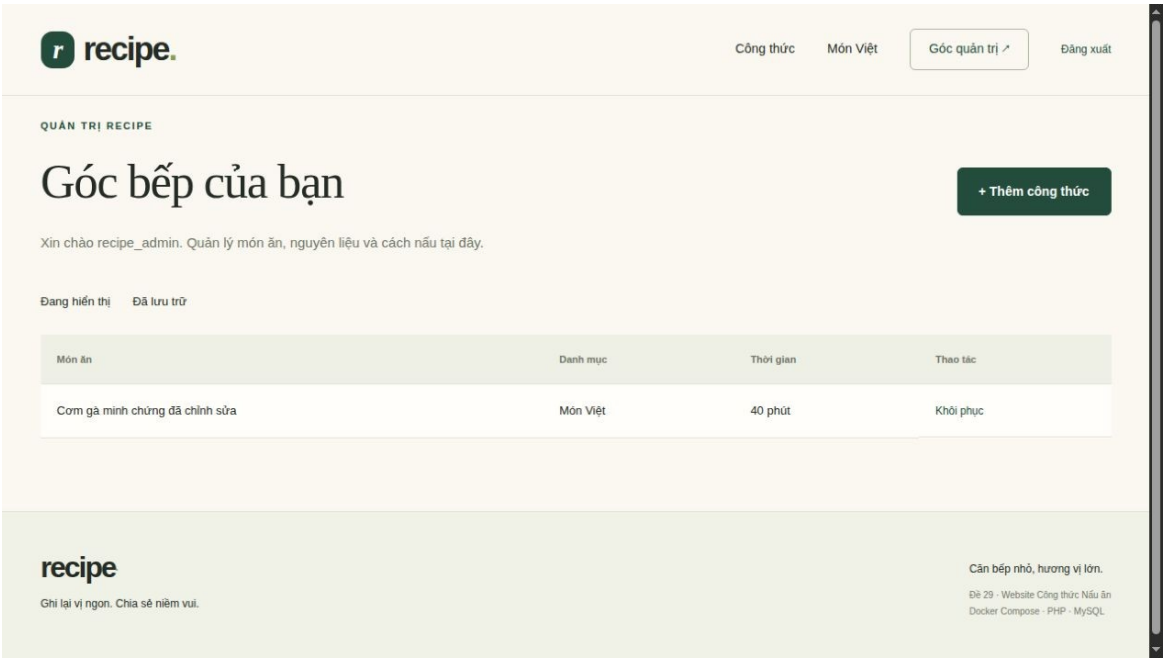
Bản ghi minh họa có ID 14. Sau khi tạo, chọn Sửa ở đúng dòng này. Đổi tên thành Cơm gà minh chứng đã chỉnh sửa và thời gian nấu thành 30 phút. Khẩu phần, nguyên liệu và các bước được giữ để đối chiếu.

Trong mã PHP, thao tác sửa UPDATE bảng recipes và thay các dòng nguyên liệu, cách làm trong một transaction. Nếu có lỗi database, dữ liệu không được cập nhật dở dang.

Kiểm tra cập nhật và lưu trữ công thức



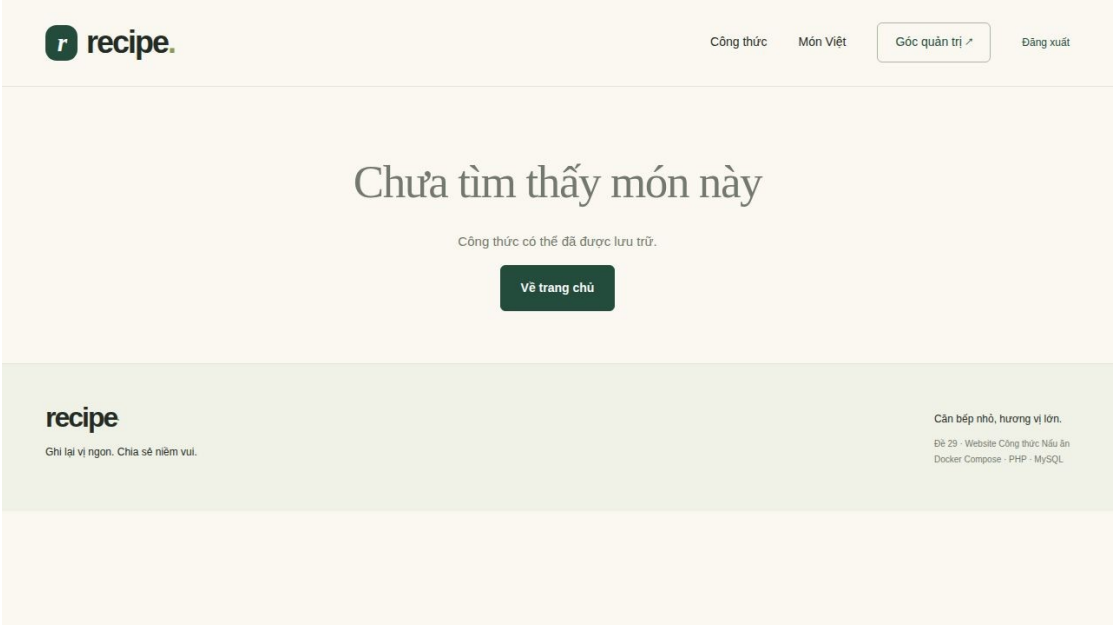
Hình 26 Tên món được cập nhật và tổng thời gian đổi thành 40 phút



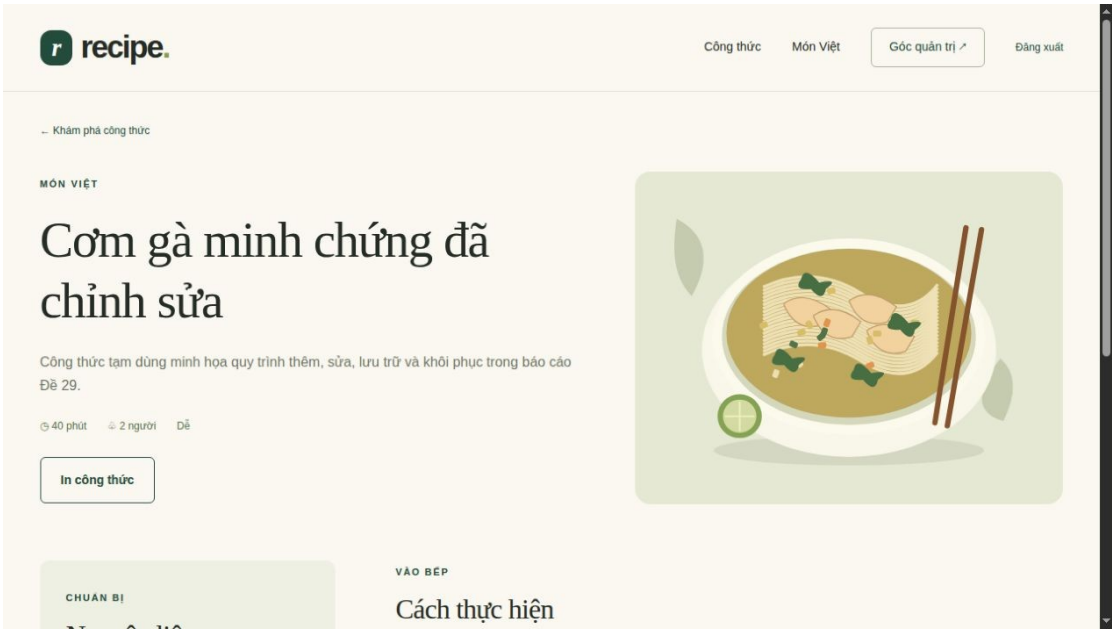
Hình 27 Bản ghi chuyển sang danh sách đã lưu trữ

Sau khi lưu, kiểm tra tên mới và tổng thời gian 10 + 30 = 40 phút. Chọn Lưu trữ rồi mở tab Đã lưu trữ. Thao tác này đặt is_deleted=1 và giữ dữ liệu trong MySQL để có thể khôi phục.

Đối chiếu trạng thái ẩn và khôi phục



Hình 28 Công thức đã lưu trữ không còn được phục vụ ở trang chi tiết



Hình 29 Cùng công thức hiển thị lại sau khi khôi phục

Khi món đã lưu trữ, URL ?recipe=14 trả trang không tìm thấy và HTTP 404. Trong danh sách lưu trữ, chọn Khôi phục để đặt is_deleted=0; tải lại URL và kiểm tra món trở lại với tên mới, 40 phút, 2 người.

Ảnh ghi nhận trạng thái khi thao tác. Sau chu trình minh chứng, bản ghi tạm ID 14 được dọn theo đúng ID và tên; database trở về sáu công thức mẫu. Các món gốc không bị dùng để thử xóa hoặc chỉnh sửa.

Cơ chế bảo vệ trong mã ứng dụng

Truy vấn và hiển thị dữ liệu

```
$s = db()->prepare('SELECT id,username,password_hash FROM users WHERE username=? AND
role=?');
$s->execute([$name, 'admin']);
htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
```

Prepared statement tách câu SQL khỏi giá trị đầu vào. Chuỗi có dấu nháy hoặc nội dung giống SQL vẫn được xử lý như dữ liệu. Escape HTML ngăn nội dung tên món và mô tả trở thành thẻ hoặc script khi xuất ra giao diện.

Session mật khẩu và CSRF

```
password_verify($password, $user['password_hash']);
session_regenerate_id(true);
hash_equals(csrf(), $_POST['csrf']);
```

Mật khẩu lưu bằng password_hash, không lưu dạng rõ trong bảng users. Đăng nhập thành công đổi session ID. Các POST như đăng nhập, lưu, lưu trữ và khôi phục đều qua kiểm tra CSRF token; token thiếu hoặc sai nhận HTTP 403. Cookie session đặt HttpOnly và SameSite=Strict.

Tình huống kiểm tra	Kết quả thực tế trong verify.py
Chưa đăng nhập mở quản trị	Chuyển về trang đăng nhập
POST thiếu CSRF	HTTP 403
Sai mật khẩu	HTTP 401 và sự kiện login_failed
Tìm chuỗi SQL injection	Được xử lý như từ khóa
Nhập nội dung chứa HTML	Nội dung được HTML escape
Đăng nhập thành công	Session ID thay đổi

Audit log ghi tên sự kiện và ID bản ghi, không ghi mật khẩu, cookie hay CSRF token. Bộ kiểm thử tự tạo dữ liệu riêng và dọn bản ghi trong khối finally để giảm ảnh hưởng tới dữ liệu mẫu.

8 Cấu hình Nginx reverse proxy

Nginx dùng hai server block: cổng nội bộ 8080 phục vụ website và 8081 phục vụ phpMyAdmin. Host publish lần lượt 127.0.0.1:8088 và 127.0.0.1:8081. Apache app và phpMyAdmin chỉ có cổng nội bộ Docker.

```
resolver 127.0.0.11 valid=10s ipv6=off;
upstream recipe_backend {
    zone recipe_backend 64k;
    server app:8080 resolve;
}
location / {
    proxy_set_header Host $http_host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_pass http://recipe_backend;
}
```

Resolver Docker và tùy chọn resolve giúp Nginx cập nhật địa chỉ app khi container được tạo lại. Các header chuyển tiếp giữ host và thông tin proxy cần thiết cho ứng dụng.

1. Sửa nginx/nginx.conf trong source.

2. Build lại image Nginx khi thay đổi cấu hình COPY vào image.

3. Chạy nginx -t trong container để xác nhận cú pháp và kiểm tra HTTP qua cổng proxy.

```
docker compose build nginx
docker compose up -d --no-deps nginx
docker compose exec -T nginx nginx -t
```

Kiểm tra cú pháp Nginx

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Hình 30 Kết quả kiểm tra cú pháp cấu hình Nginx

Security headers và chặn đường dẫn nhạy cảm

Kết quả HTTP và security headers

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
GET http://127.0.0.1:8088/ -> HTTP 200
Server: nginx
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self' data:;
connect-src 'self'; frame-ancestors 'self'; base-uri 'self'; form-action 'self'
GET /.env -> HTTP 403
GET /?recipe=999999 -> HTTP 404
GET /?page=health -> HTTP 200
```

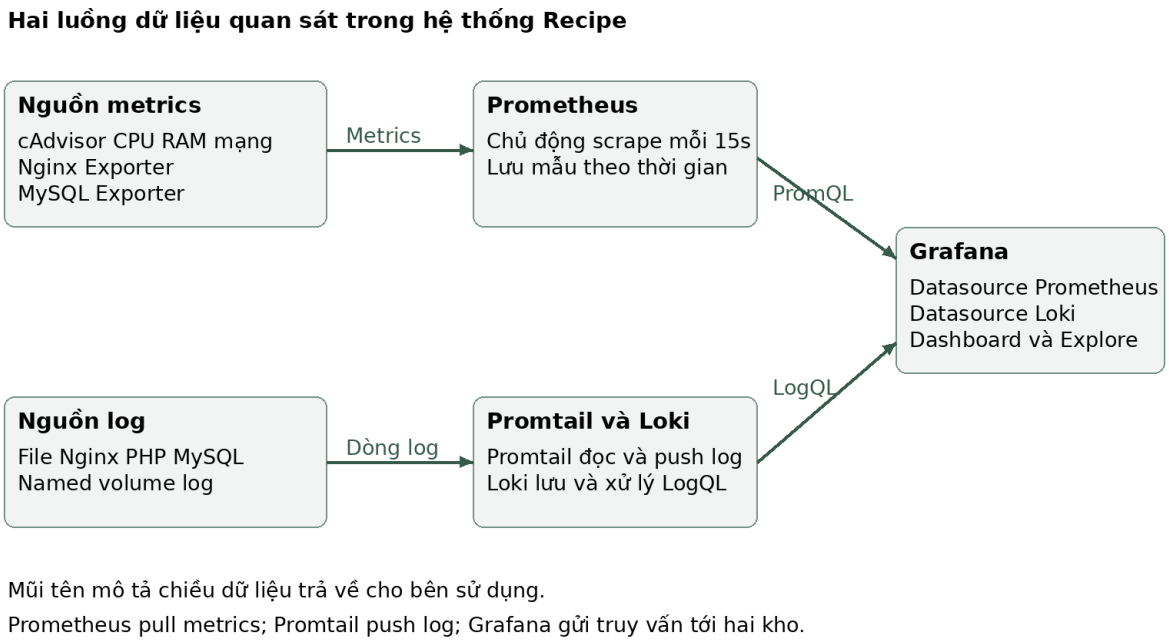
Hình 31 HTTP 200 403 404 và security headers từ Nginx

Header	Tác dụng
X-Content-Type-Options	nosniff hạn chế đoán sai kiểu nội dung
X-Frame-Options	SAMEORIGIN hạn chế nhúng trang từ nguồn khác
Referrer-Policy	Giới hạn thông tin referrer gửi tới nguồn khác
Permissions-Policy	Tắt camera microphone và geolocation không cần dùng
Content-Security-Policy	Giới hạn các nguồn script style ảnh kết nối form

Nginx ẩn số phiên bản, chặn file cấu hình như /.env và giới hạn request đăng nhập theo địa chỉ client. Bản lab chọn security headers theo phương án đề bài cho phép; không mô tả website localhost HTTP như một triển khai HTTPS công khai.

Access log dùng JSON gồm time, method, uri, status, bytes, request_time và upstream. Trường uri không kèm query string để giảm việc lưu dữ liệu người dùng vào log.

9 Luồng giám sát bằng Prometheus và Grafana



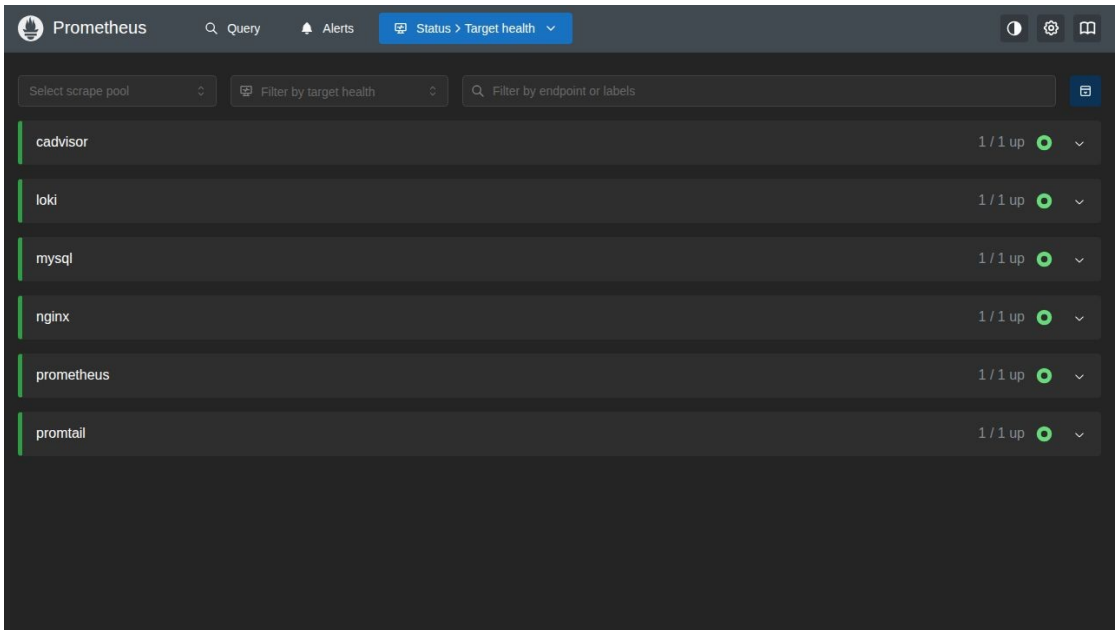
Hình 32 Luồng metrics và log cùng được truy vấn qua Grafana

Prometheus chủ động gọi endpoint metrics theo scrape_interval 15 giây và lưu mẫu kèm timestamp. cAdvisor cung cấp CPU, RAM và mạng container; Nginx Exporter đọc stub_status; MySQL Exporter dùng tài khoản recipe_exporter để đọc số liệu database.

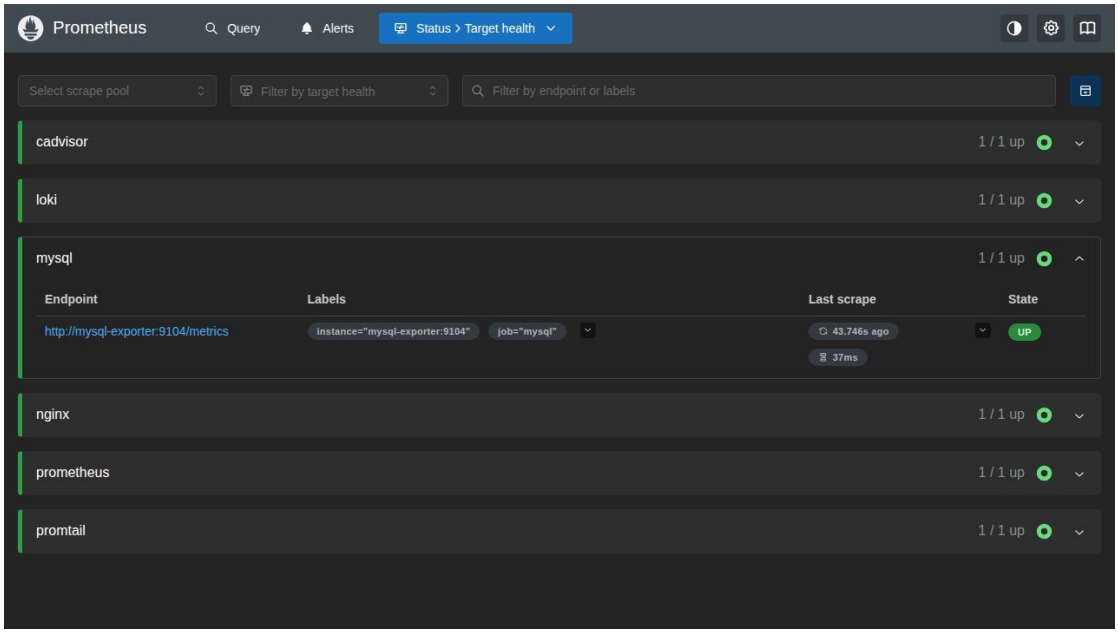
Grafana không tự thu CPU từ Docker. Mỗi panel chỉ định datasource và biểu thức PromQL; Grafana gửi yêu cầu tới Prometheus cho khoảng thời gian được chọn rồi dùng kết quả để vẽ biểu đồ. Refresh 10 giây của dashboard không làm chu kỳ scrape 15 giây thay đổi.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
scrape_configs:
  - job_name: nginx
    static_configs: [{targets: ["nginx-exporter:9113"]}]
  - job_name: mysql
    static_configs: [{targets: ["mysql-exporter:9104"]}]
```

Kiểm tra target và endpoint metrics



Hình 33 Sáu nhóm target Prometheus đều có trạng thái UP

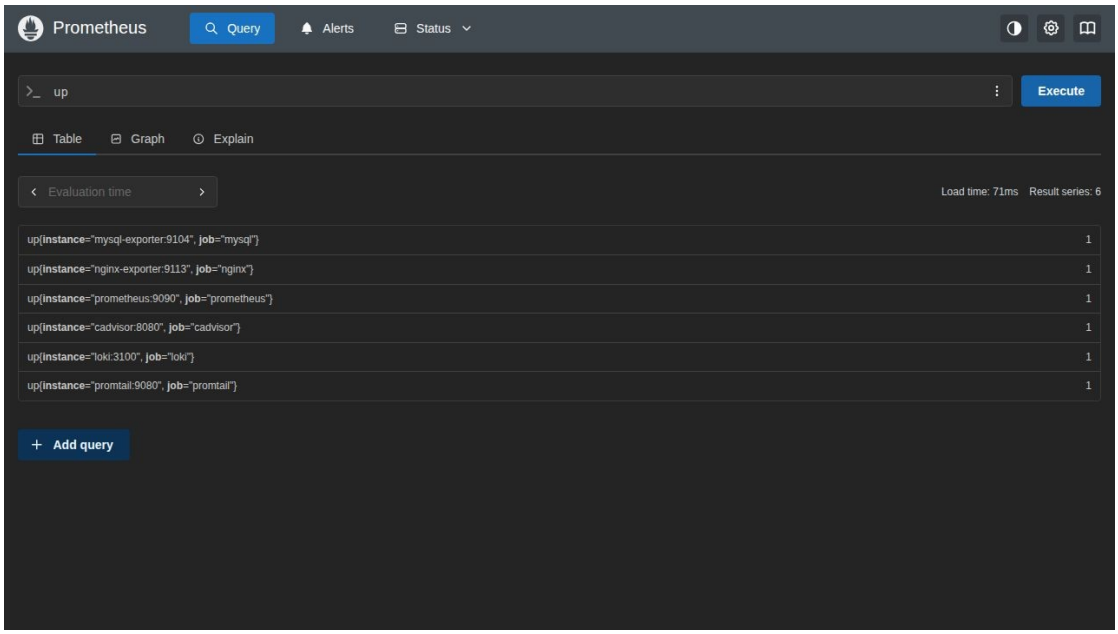


Hình 34 Endpoint MySQL Exporter và nhãn job instance

Mở Prometheus tại :9090/targets. Sáu target là prometheus, cadvisor, nginx, mysql, loki và promtail. Mở nhóm mysql để xem endpoint mysql-exporter:9104/metrics, lần scrape và trạng thái.

Sáu target không đồng nghĩa sáu container: một endpoint cAdvisor cung cấp metrics của mười một container. up=1 chứng minh scrape endpoint thành công; kiểm tra mysql_up và nginx_up mới xác minh exporter đọc được dịch vụ phía sau.

Truy vấn PromQL và đối chiếu API



Hình 35 Câu up trả sáu chuỗi dữ liệu có giá trị một

Target metrics từ API Prometheus

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

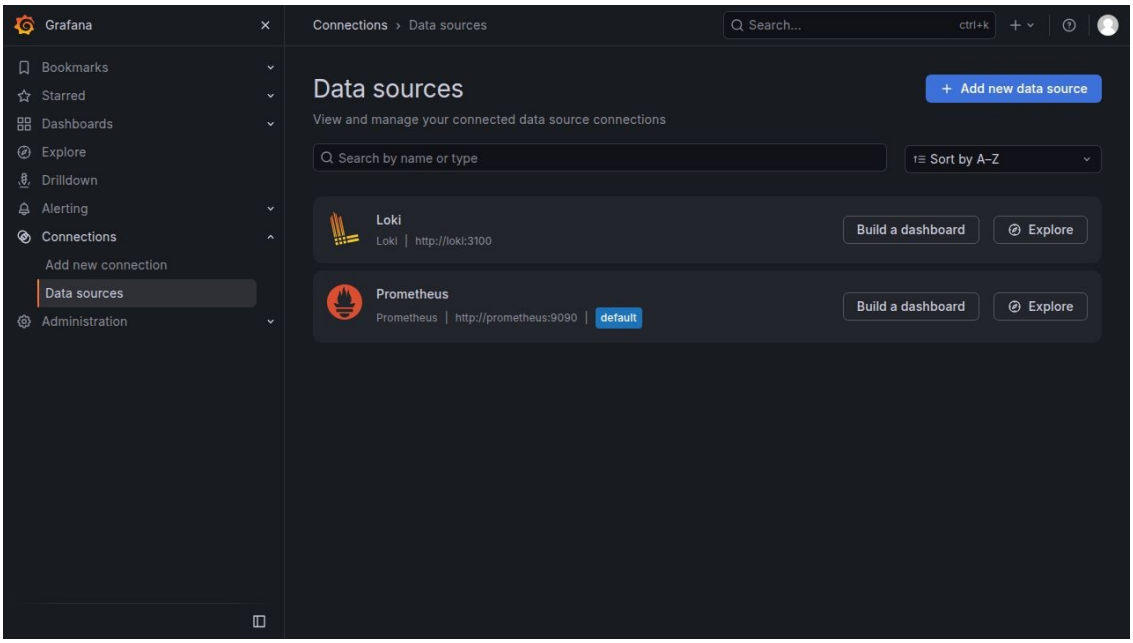
```
JOB          HEALTH    SCRAPE URL
cadvisor     up        http://cadvisor:8080/metrics
loki         up        http://loki:3100/metrics
mysql        up        http://mysql-exporter:9104/metrics
nginx        up        http://nginx-exporter:9113/metrics
prometheus   up        http://prometheus:9090/metrics
promtail     up        http://promtail:9080/metrics
```

Hình 36 API Prometheus trả endpoint và health của sáu target

```
up
sum(up)
up{job="mysql"}
mysql_up
nginx_up
```

up trả một chuỗi cho mỗi target với nhãn job và instance. sum(up) cộng các target scrape thành công. Nếu up của mysql bằng 1 nhưng mysql_up bằng 0, exporter vẫn trả metrics nhưng không kết nối được MySQL. Tình huống này cần kiểm tra database, tài khoản exporter và đường mạng db_net.

Provisioning nguồn dữ liệu Grafana



Hình 37 Hai datasource Loki và Prometheus được khai báo sẵn

```
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
  - name: Loki
    uid: loki
    type: loki
    access: proxy
    url: http://loki:3100
```

Datasource nằm trong grafana/provisioning/datasources. Các URL trên dùng tên dịch vụ Docker, khác với URL localhost người dùng mở trên Ubuntu. Dashboard JSON và cấu hình provider được mount chỉ đọc để nạp tự động khi Grafana khởi động.

Đăng nhập Grafana bằng GRAFANA_ADMIN_USER và GRAFANA_ADMIN_PASSWORD trong .env. Bộ kiểm thử gọi health API của cả hai datasource và nhận thông báo kết nối thành công.

Đọc dashboard trạng thái CPU và RAM



Hình 38 Dashboard hiển thị sáu target UP cùng CPU và RAM của container

Panel hoặc số liệu	Cách hiểu
Targets đang UP bằng 6	Sáu endpoint metrics được scrape thành công
MySQL kết nối bằng 1	Exporter kết nối được database
Nginx hoạt động bằng 1	Exporter đọc được stub_status
Kết nối MySQL	Tổng kết nối hiện mở gồm cả client giám sát
CPU theo container	Phần trăm công suất một CPU logic theo dịch vụ
RAM theo container	Working set hiển thị theo đơn vị byte MiB GiB

Đường màu phân biệt app, db, grafana và các dịch vụ khác. Biểu đồ cho thấy hoạt động tăng trong lúc thực hiện truy vấn và chụp minh chứng. Một mẫu tăng chưa đủ kết luận lỗi; cần xem mức kéo dài, giới hạn RAM và log cùng thời điểm.

Giải thích truy vấn CPU RAM và đơn vị đo

CPU trung bình theo dịch vụ

```
sum by (container_label_com_docker_compose_service) (
  rate(container_cpu_usage_seconds_total{
    container_label_com_docker_compose_project="recipe",
    image!=""
  }[2m])
) * 100
```

Metric CPU là bộ đếm thời gian CPU tích lũy. rate chuyển mức tăng trong cửa sổ hai phút thành giây CPU mỗi giây. Bộ lọc project chỉ chọn Recipe; sum by gom theo tên dịch vụ Compose. Nhân 100 biểu diễn phần trăm của một CPU logic.

Ví dụ dùng 60 giây CPU trong 120 giây tương đương khoảng 50 phần trăm. Trên máy nhiều CPU, giá trị có thể vượt 100 phần trăm. rate xử lý trường hợp counter đặt lại khi tiến trình khởi động lại.

RAM working set theo dịch vụ

```
sum by (container_label_com_docker_compose_service) (
  container_memory_working_set_bytes{
    container_label_com_docker_compose_project="recipe",
    image!=""
  }
)
```

RAM là giá trị hiện tại nên đọc trực tiếp, không tính rate. Metric trả byte; chia 1024 rồi 1024 để đổi sang MiB. Ví dụ 104857600 byte tương đương 100 MiB. Working set có thể gồm một phần cache, không được đồng nhất với RSS.

Khoảng hiển thị và cửa sổ tính toán

Chọn Last 1 hour xác định chiều dài biểu đồ. Cửa sổ [2m] nằm trong công thức và áp dụng tại từng điểm trên biểu đồ. Vì vậy biểu đồ một giờ vẫn có thể sử dụng tốc độ trung bình hai phút cho mỗi điểm.

Theo dõi Nginx MySQL và lưu lượng mạng



Hình 39 Bốn panel request kết nối MySQL và dữ liệu mạng

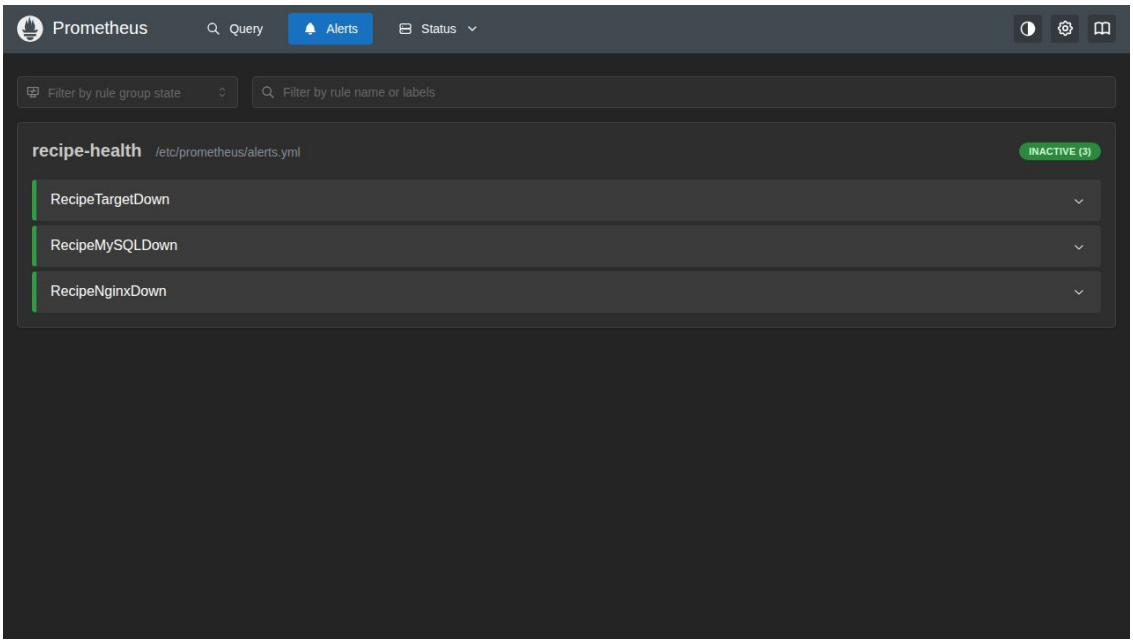
```
rate(nginx_http_requests_total[2m])
nginx_connections_active
mysql_global_status_threads_connected
rate(mysql_global_status_questions[2m])
```

Request mỗi giây dùng bộ đếm Nginx; số kết nối Nginx và MySQL là giá trị hiện tại. Questions mỗi giây là tốc độ lệnh từ client gửi tới MySQL, có thể bao gồm phpMyAdmin và exporter. Nếu bộ đếm request tăng 240 trong 120 giây, tốc độ xấp xỉ 2 request mỗi giây.

```
sum by (container_label_com_docker_compose_service) (
  rate(container_network_receive_bytes_total{
    container_label_com_docker_compose_project="recipe",
    interface!="lo"
  }[2m])
)
```

Truy vấn mạng trả byte nhận mỗi giây, bỏ giao diện loopback. Grafana định dạng thành kB/s hoặc đơn vị phù hợp. Các số đo hạ tầng được dùng để khoanh vùng, không thay thế kiểm thử nghiệp vụ.

Luật cảnh báo Prometheus



Hình 40 Ba luật cảnh báo đã nạp và đang ở trạng thái INACTIVE

Tên cảnh báo	Biểu thức	Điều kiện thời gian
RecipeTargetDown	up == 0	Duy trì một phút
RecipeMySQLDown	mysql_up == 0	Duy trì một phút
RecipeNginxDown	nginx_up == 0	Duy trì một phút

```
alert: RecipeMySQLDown
expr: mysql_up == 0
for: 1m
labels:
  severity: critical
```

Prometheus đánh giá rule mỗi 15 giây. Khi điều kiện xuất hiện, rule có thể ở trạng thái pending; nếu kéo dài đủ for: 1m mới thành firing. Khi điều kiện hết, cảnh báo trở về trạng thái không kích hoạt.

Ảnh chụp INACTIVE cho thấy tại thời điểm đó các điều kiện lỗi không xảy ra. Cấu hình hiện chưa nối Alertmanager hoặc contact point gửi email; vì vậy không tuyên bố đã có thông báo tự động ra dịch vụ ngoài.

10 Cấu hình Loki Promtail và nguồn log

Nginx, PHP và MySQL ghi log vào named volume riêng. Promtail chỉ đọc các volume này, gắn nhãn và gửi log tới Loki. Volume positions giữ vị trí đã đọc. Loki lưu dữ liệu TSDB và chunks trên filesystem trong volume, retention cấu hình 168 giờ.

```
positions:
  filename: /positions/positions.yaml
clients:
  - url: http://loki:3100/loki/api/v1/push
scrape_configs:
  - job_name: nginx-access
    static_configs:
      - targets: [localhost]
        labels:
          project: recipe
          job: nginx
          service: nginx
          __path__: /logs/nginx/access.log
```

Nguồn	Đường dẫn Promtail	Nhãn job
Nginx access	/logs/nginx/access.log	nginx
Nginx error	/logs/nginx/error.log	nginx_error
Ứng dụng PHP	/logs/app/*.log	app
MySQL error	/logs/mysql/*.log	mysql

Prometheus dùng pull để lấy metrics. Promtail dùng push để gửi log. Grafana truy vấn Prometheus bằng PromQL và Loki bằng LogQL. Promtail không được cấp Docker socket; phạm vi đọc log giới hạn ở các volume đã chỉ định.

Promtail được giữ để đáp ứng yêu cầu môn học. Tài liệu Grafana ghi nhận Promtail hết hỗ trợ ngày 02/03/2026; khi phát triển hệ thống vận hành lâu dài cần chuyển sang Alloy hoặc client được hỗ trợ.

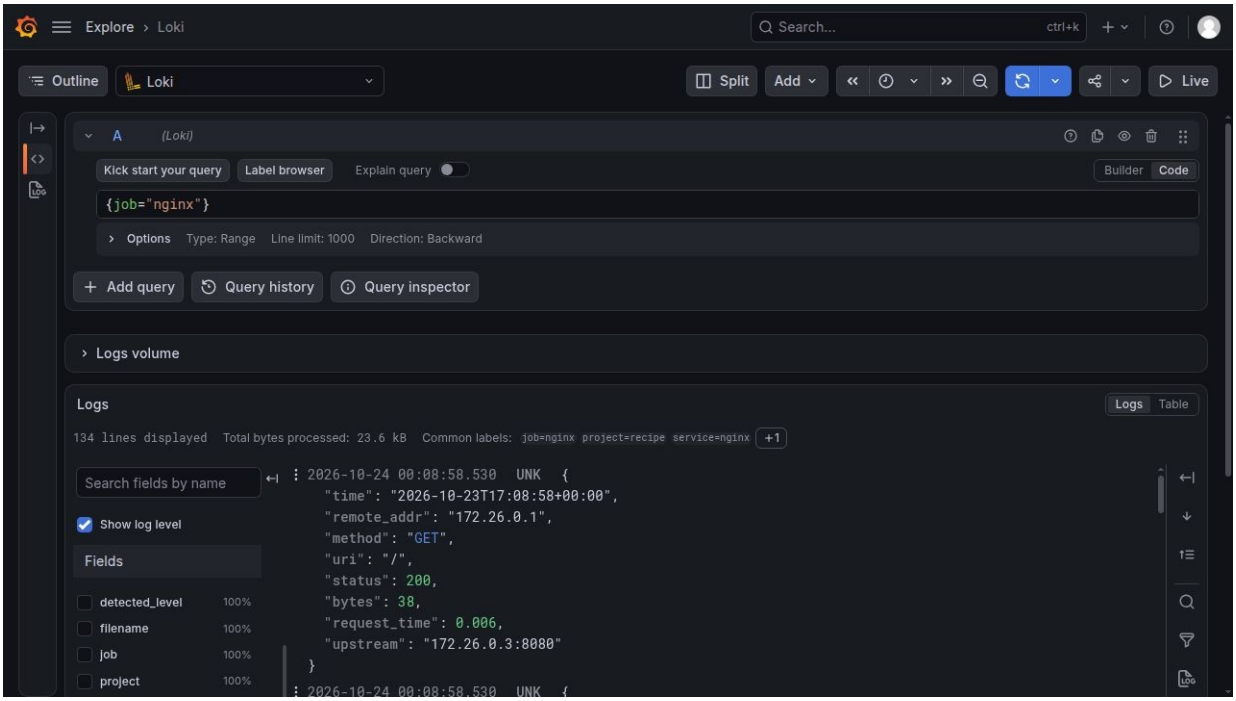
Truy vấn LogQL xem access log Nginx

1. Mở Grafana Explore và chọn datasource Loki.

2. Chuyển sang Code, chọn Last 1 hour và nhập truy vấn dưới đây.

3. Chạy truy vấn; đối chiếu nhãn nguồn, timestamp và nội dung dòng log.

```
{job="nginx"}
```



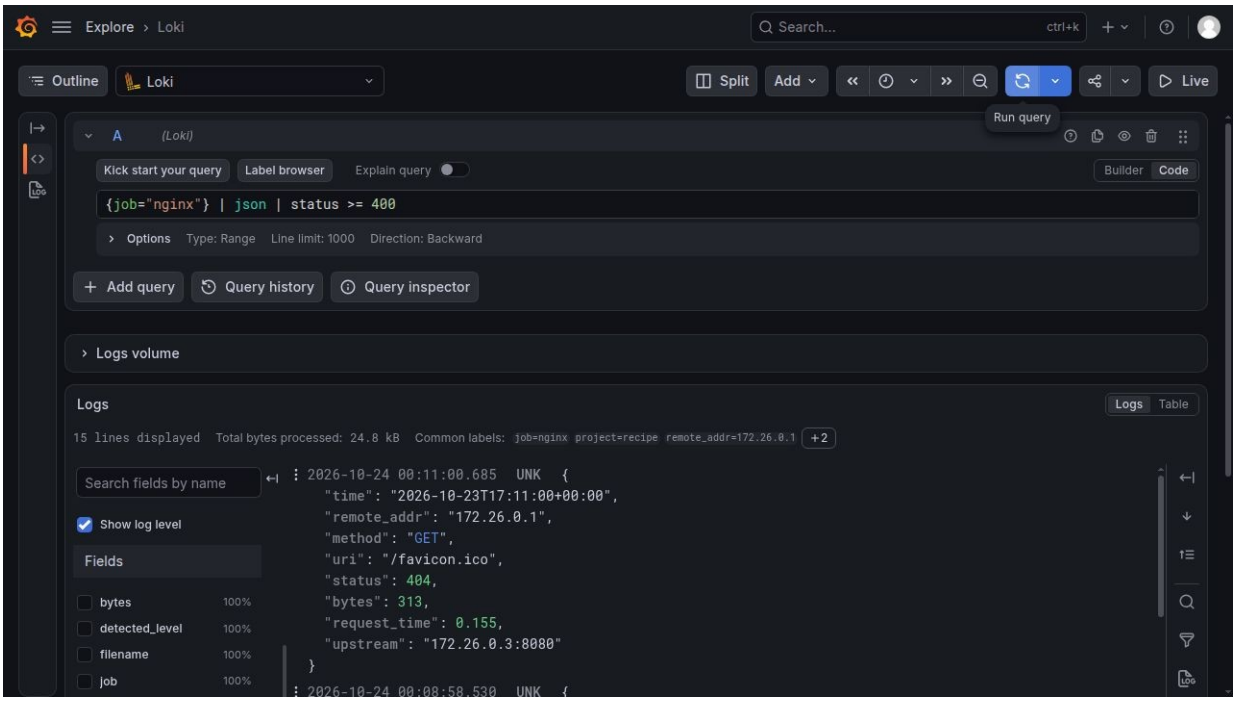
Hình 41 Explore trả các dòng access log và nhãn nguồn Nginx

Phần trong dấu ngoặc nhọn chọn luồng log có nhãn job=nginx. Mỗi dòng có thời điểm, phương thức HTTP, URI, mã trạng thái, số byte, thời gian xử lý và upstream. Nhãn project=recipe giúp phân biệt dự án khi cùng Loki thu nhiều hệ thống.

Loki tổ chức log theo timestamp và labels. Phân tích JSON trong câu truy vấn diễn ra khi đọc; cấu hình Promtail hiện chỉ gắn nhãn nguồn, không tạo riêng một nhãn chỉ mục cho từng URI hoặc địa chỉ client.

Truy vấn LogQL lọc lỗi HTTP

```
{job="nginx"} | json | status >= 400
```



Hình 42 Các request lỗi được lọc bằng trường status sau khi parse JSON

1. Mở một URL món không tồn tại như ?recipe=999999 để tạo HTTP 404.

2. Truy cập /.env để kiểm tra Nginx từ chối bằng HTTP 403.

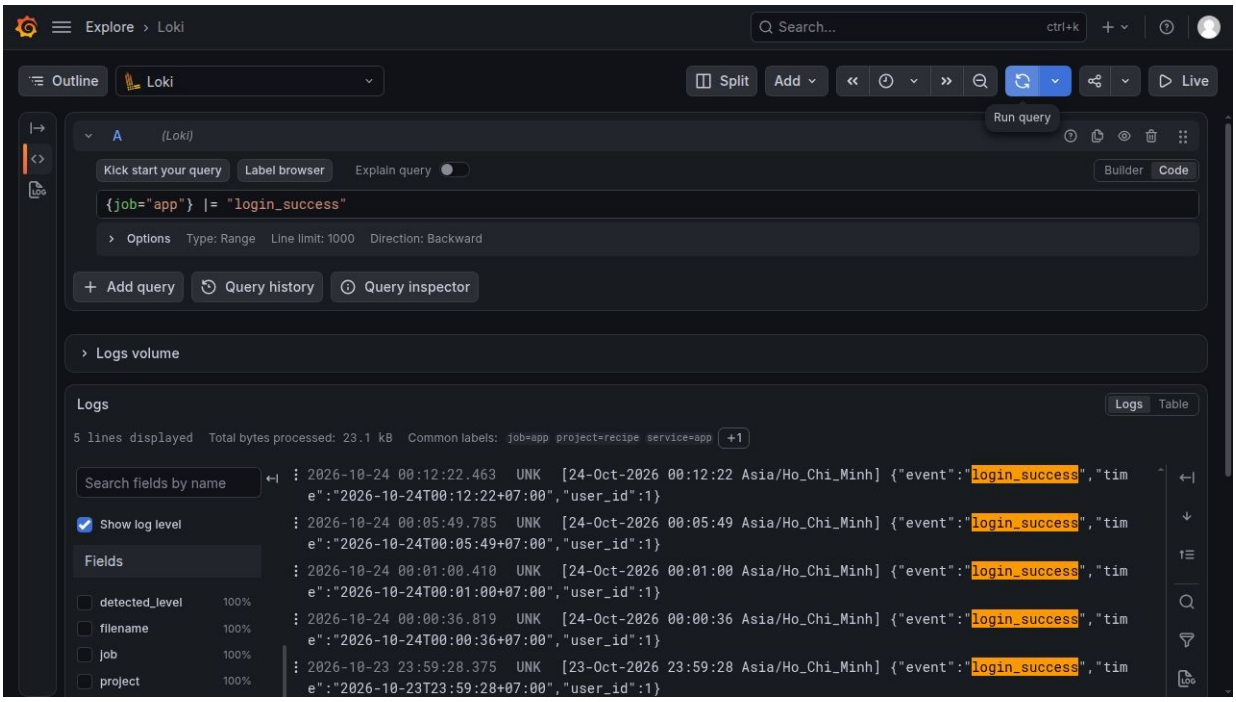
3. Quay lại Explore, chạy truy vấn trên và kiểm tra các dòng trong thời gian vừa thử.

json tách nội dung log thành trường có tên. status >= 400 chỉ giữ các request có mã từ 400 trở lên. Mã 403 là từ chối truy cập, 404 là không tìm thấy, 500 là lỗi phía máy chủ và 502 thường liên quan trao đổi giữa proxy với upstream.

Ảnh có thể chứa cả 404 của favicon hoặc đường dẫn khác; không suy ra mọi dòng 404 đều do cùng một thao tác thử. Muốn khoanh vùng, đối chiếu timestamp, uri và mã status.

Truy vấn LogQL theo dõi thao tác quản trị

```
{job="app"} |= "login_success"
```



Hình 43 Các sự kiện đăng nhập thành công có timestamp và user_id

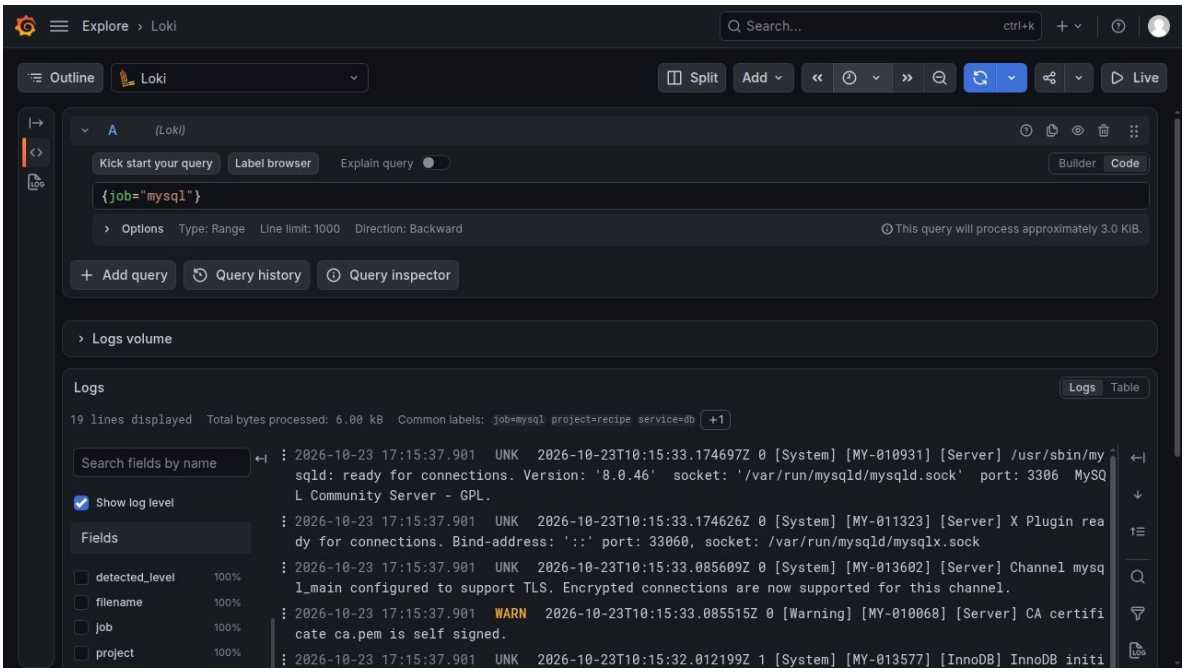
Toán tử |= giữ dòng chứa đoạn văn bản chỉ định. login_success do ứng dụng ghi sau khi xác minh mật khẩu và tạo phiên đăng nhập. Log có tiền tố PHP trước nội dung JSON, nên truy vấn chuỗi là cách trực tiếp để lọc sự kiện trong cấu hình hiện tại.

Chuỗi lọc	Sự kiện ứng dụng
login_success	Đăng nhập thành công
login_failed	Đăng nhập thất bại
recipe_saved	Thêm hoặc sửa công thức
recipe_archived	Lưu trữ công thức
recipe_restored	Khôi phục công thức

Dashboard log dùng {job="app"} |= "event" để xem nhiều loại sự kiện cùng lúc. Các truy vấn theo tên sự kiện dùng khi cần kiểm tra một thao tác cụ thể.

Truy vấn log MySQL và thống kê từ log

```
{job="mysql"}
```



Hình 44 MySQL error log hiển thị thông báo khởi động và sẵn sàng kết nối

Nguồn này là error log MySQL, không phải toàn bộ câu SQL đã chạy. Một database khỏe có thể không ghi thêm error log trong nhiều giờ. Vì vậy ảnh chọn Last 7 days để thấy thông báo khởi động đã lưu; test cũng dùng cửa sổ bảy ngày cho nguồn này thay vì bắt buộc phải có log mới trong một giờ.

Đếm request theo mã HTTP trong năm phút

```
sum by (status) (
  count_over_time(
    {job="nginx"} | json | __error__="" [5m]
  )
)
```

count_over_time đếm dòng trong cửa sổ năm phút; sum by status gom theo mã HTTP. Điều kiện __error__="" bỏ dòng lỗi phân tích. Đây là truy vấn sinh số liệu từ log, có thể đưa vào panel biểu đồ. Không có dòng khớp trong khoảng chọn có thể cho kết quả rỗng thay vì số 0.

11 Hardening được kiểm chứng tại runtime

Trích các trường hardening từ Docker inspect

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

SERVICE	UID	RO	CAP_DROP	NNP	PRIVILEGED
app	33:33	True	ALL	True	False
cadvisor	0	True	ALL	True	False
db	999:999	True	ALL	True	False
grafana	472:472	True	ALL	True	False
loki	10001:10001	True	ALL	True	False
mysql-exporter	65534:65534	True	ALL	True	False
nginx	101:101	True	ALL	True	False
nginx-exporter	65534:65534	True	ALL	True	False
phpmyadmin	33:33	True	ALL	True	False
prometheus	65534:65534	True	ALL	True	False
promtail	10001:10001	True	ALL	True	False

Hình 45 Các trường UID read only capabilities và privileged từ Docker inspect

UID thực tế của các dịch vụ

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uid=101(nginx) gid=101(nginx) groups=101(nginx)
uid=999(mysql) gid=999(mysql) groups=999(mysql)
```

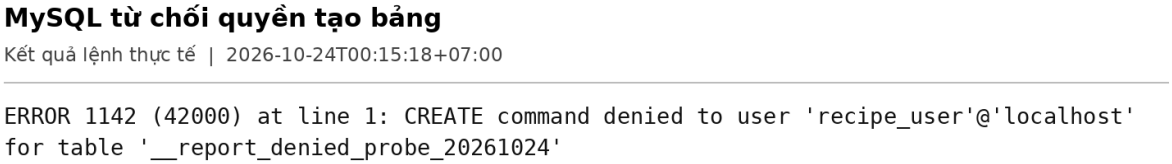
Hình 46 Lệnh id xác nhận UID của app Nginx và MySQL

App và phpMyAdmin dùng UID 33, Nginx 101, MySQL 999, Grafana 472, Prometheus và exporter 65534, Loki và Promtail 10001. Tất cả service bỏ ALL capabilities, bật no-new-privileges và không privileged.

Root filesystem chỉ đọc, trong khi named volume và tmpfs cung cấp các vùng ghi cần thiết. Kiểm tra runtime quan trọng vì file YAML chỉ cho biết cấu hình dự kiến; Docker inspect cho biết các thuộc tính đã thực sự áp dụng cho container.

cAdvisor là ngoại lệ chạy root để đọc cgroups và metadata runtime. Mount host chỉ đọc và không publish cổng làm giảm phạm vi tiếp cận, nhưng socket Unix gắn :ro vẫn cho phép gọi API runtime; collector phải được tin cậy.

Phân quyền database và bảo vệ bí mật



Hình 47 MySQL trả mã 1142 khi recipe_user thử tạo bảng

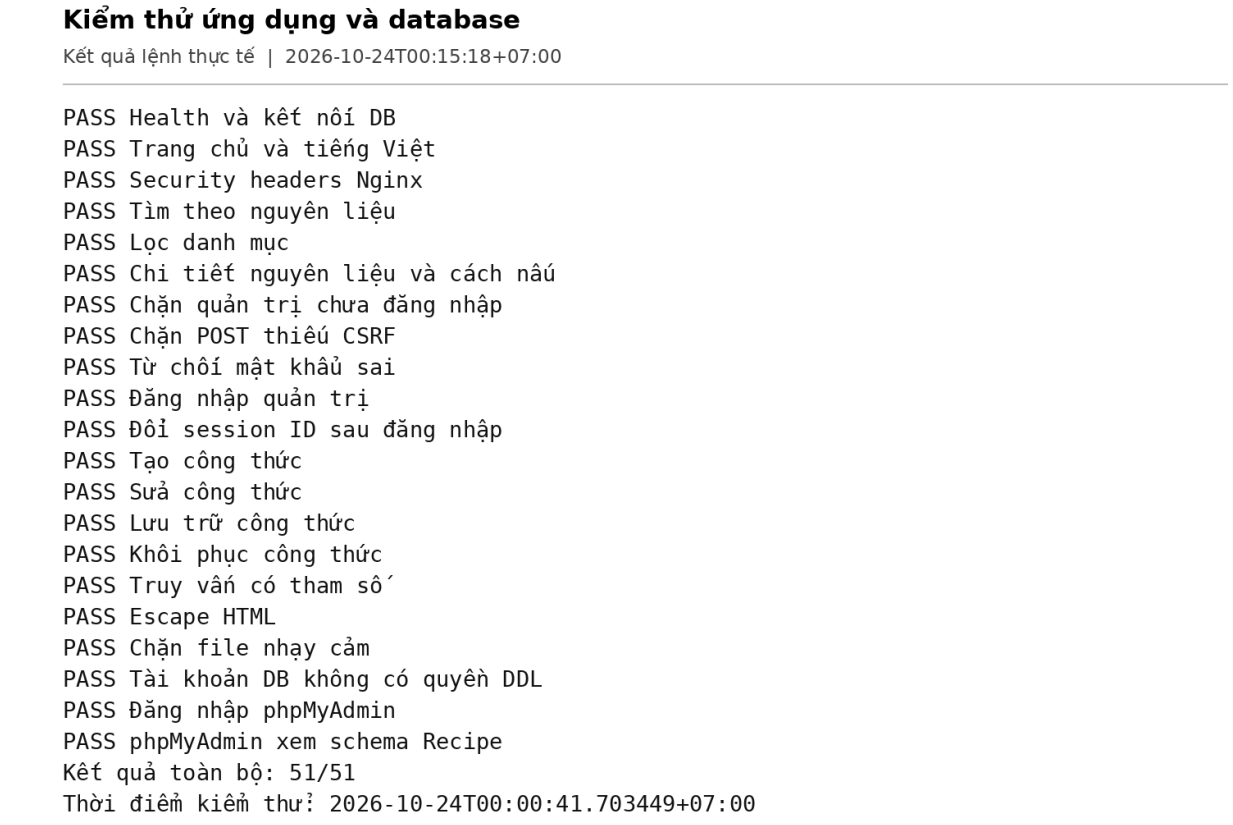
Biện pháp	Cách triển khai và kiểm tra
Quyền tài khoản ứng dụng	Chỉ SELECT INSERT UPDATE DELETE trên recipe_db; SHOW GRANTS và thử CREATE TABLE bị từ chối
Tài khoản giám sát riêng	recipe_exporter có quyền đọc phục vụ metrics và tối đa ba kết nối
Bí mật cục bộ	Mật khẩu ngẫu nhiên, .env mode 0600, secrets mode 0700, git check-ignore xác nhận loại khỏi Git
Tách mạng	db_net và monitoring_net internal; MySQL không publish cổng
Giới hạn tài nguyên	Memory limit từng service; Docker stdout log rotation 10 MB nhân ba file
Cấu hình ổn định	Image bên ngoài ghim digest; Grafana không tự cài và cập nhật plugin khi chạy

phpMyAdmin chặn đăng nhập root và sử dụng tài khoản ứng dụng để quản lý các bản ghi. Script setup dùng quyền quản trị để nạp schema trong giai đoạn triển khai; website không giữ quyền tạo bảng.

Ảnh cổng và quyền file ở phần mạng ghi nhận các UI đều bind 127.0.0.1. Báo cáo không hiển thị nội dung .env hoặc hash mật khẩu của bảng users.

12 Kiểm thử chức năng ứng dụng và database

Chạy python3 scripts/verify.py sau khi toàn bộ dịch vụ đã khởi động. Script thực hiện request thật, tạo bản ghi tạm, kiểm tra kết quả và dọn bản ghi trong finally. Kết quả được lưu vào minh-chung/verification.json và verification.txt.



Hình 48 Nhóm kiểm thử HTTP tìm kiếm CRUD xác thực và database

Chu trình CRUD kiểm tra cả giao diện công khai lẫn thay đổi trạng thái trong database. Tình huống bảo mật thử CSRF, mật khẩu sai, SQL injection và HTML escape. Kiểm tra phpMyAdmin đăng nhập bằng tài khoản ứng dụng và đọc được năm bảng chính.

Kết quả hiện tại là 51/51 đạt tại 00:00:41 ngày 24/10/2026. Đây là phạm vi kiểm thử của bản lab, không thay thế kiểm toán bảo mật hoặc thử tải cho một hệ thống vận hành lớn.

Kiểm thử giám sát log và hardening

Kiểm thử monitoring và LogQL

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
PASS Prometheus targets
PASS Metrics mysql_up
PASS Metrics nginx_up
PASS Metrics count(container_cpu_usage_seconds_total{container_label_com_docker_compos
e_project="recipe"})
PASS Metrics count(container_memory_working_set_bytes{container_label_com_docker_compo
se_project="recipe"})
PASS Grafana datasource prometheus
PASS Grafana datasource loki
PASS Dashboard tự provision
PASS LogQL {job="nginx"}
PASS LogQL {job="nginx"} | json | status >= 400
PASS LogQL {job="app"} |= "login_success"
PASS LogQL {job="mysql"}
Kết quả toàn bộ: 51/51
Thời điểm kiểm thử: 2026-10-24T00:00:41.703449+07:00
```

Hình 49 Kết quả kiểm thử target metrics datasource dashboard và bốn LogQL

Kiểm thử runtime và network

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
PASS Runtime hardening app
PASS Runtime hardening cadvisor
PASS Runtime hardening db
PASS MySQL không publish cổng
PASS Runtime hardening grafana
PASS Loopback grafana 3000/tcp
PASS Runtime hardening loki
PASS Loopback loki 3100/tcp
PASS Runtime hardening mysql-exporter
PASS Runtime hardening nginx
PASS Loopback nginx 8080/tcp
PASS Loopback nginx 8081/tcp
PASS Runtime hardening nginx-exporter
PASS Runtime hardening phpmyadmin
PASS Runtime hardening prometheus
PASS Loopback prometheus 9090/tcp
PASS Runtime hardening promtail
PASS Network isolation DB
Kết quả toàn bộ: 51/51
Thời điểm kiểm thử: 2026-10-24T00:00:41.703449+07:00
```

Hình 50 Kết quả kiểm thử cấu hình runtime và cổng mạng

Bộ kiểm thử xác nhận sáu target UP, metrics CPU/RAM của mười một container, hai datasource healthy, dashboard metrics có mười panel và bốn LogQL trả dữ liệu. Các kiểm tra runtime xác nhận UID theo vai trò, read_only, cap_drop ALL, no-new-privileges, loopback và db_net internal.

Kiểm tra cài mới và sao lưu database

Kết quả cài mới độc lập đã lưu

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
Thời điểm gốc: 2026-10-23T17:13:36.037522+07:00
Nguồn: GitHub clean clone
Commit: 9393eaa0aa5daa03ba9aba797e03819f4ee04e33
Project độc lập: recipe-repro

PASS Fresh app database connection
PASS Fresh seed and UTF8
PASS Fresh administrator login
PASS Fresh schema has exactly 5 application tables
PASS Fresh Grafana Prometheus provisioning
PASS Fresh Grafana Loki provisioning
PASS Fresh 6 Prometheus targets
PASS Fresh 11 services running

TOTAL 8/8; bảng ứng dụng: 5
```

Hình 51 Tám kiểm tra cài mới độc lập từ GitHub ở commit đã ghi nhận

Bản sao lưu database mới

Kết quả lệnh thực tế | 2026-10-24T00:15:18+07:00

```
/home/longphan/DTC245220017-De 29/runtime/backups/recipe-20261024-001522.sql.gz
File mode: 0o600
Size bytes: 24959
Gzip can be read: True
Git ignored: True
```

Hình 52 Bản sao lưu mới có quyền 0600 và đọc được định dạng gzip

Kiểm tra cài mới thực hiện ngày 23/10/2026 tại commit 9393eaa, project recipe-repro, volume và cổng khác hệ thống chính. Kết quả có đúng năm bảng, sáu công thức tiếng Việt, admin đăng nhập được, hai datasource hoạt động và đủ dịch vụ. Các container thử được dọn sau kiểm tra.

Bản sao lưu mới được tạo trong lần bổ sung báo cáo bằng scripts/backup.py. File gzip nằm trong runtime/backups, được Git bỏ qua. Kiểm tra đọc gzip xác nhận file có dữ liệu; chưa thực hiện phục hồi ghi đè database đang chạy trong đợt cập nhật báo cáo này.

13 Vận hành và các vấn đề đã xử lý

```
docker compose up -d
docker compose ps -a
docker compose logs --tail=50 app nginx
python3 scripts/backup.py
docker compose stop
```

Vấn đề gặp khi triển khai	Nguyên nhân và cách xử lý đã áp dụng
Container chưa áp dụng cổng mới	Tạo lại đúng service để Docker áp dụng PortBindings từ Compose
Ứng dụng cũ theo WordPress	Chuyển sang PHP theo khung mẫu; lưu bản sao và giữ bảng volume cũ
Ghi file trên root filesystem chỉ đọc	Chuẩn bị thư mục và quyền trong image; dùng volume hoặc tmpfs cho dữ liệu ghi
Nginx giữ địa chỉ app cũ	Dùng Docker resolver và upstream resolve để cập nhật DNS
Grafana cập nhật plugin khi runtime chỉ đọc	Tắt tự preinstall và update, giữ datasource được provision
Grafana thiếu RAM khi mở dashboard	Tăng giới hạn từ 384 MiB lên 768 MiB, đặt GOMEMLIMIT và giảm plugin không dùng
MySQL error log không có dòng trong một giờ	Database không phát sinh lỗi mới; chọn cửa sổ bảy ngày và sửa false failure của test

Những mục trên mô tả vấn đề đã gặp trong quá trình thực hiện và cấu hình hiện đã áp dụng. Ảnh trạng thái, headers, dashboard và các trường runtime ở các phần trước chứng minh kết quả sau xử lý; không dựng lại ảnh lỗi cũ để thay bằng chứng.

Khi gặp No data, kiểm tra datasource, khoảng thời gian, nhãn và thời điểm phát sinh sự kiện trước khi kết luận dịch vụ hỏng. Với biểu đồ rate, cần có đủ mẫu scrape trong cửa sổ tính.

Kịch bản demo giới hạn và tài liệu tham khảo

1. Mở GitHub, giới thiệu source và ba giai đoạn commit kỹ thuật.

2. Chạy docker compose ps, mở website, tìm nguyên liệu và lọc danh mục.

3. Xem chi tiết món rồi đăng nhập quản trị, minh họa thêm sửa lưu trữ khôi phục bằng dữ liệu riêng.

4. Mở phpMyAdmin đối chiếu sáu công thức bằng SELECT JOIN.

5. Mở Prometheus Targets và Grafana, giải thích sáu target và mười một container.

6. Chạy các LogQL Nginx, lỗi HTTP, login_success và MySQL; giải thích time range.

7. Chỉ ra hardening từ Docker inspect, quyền DB và cổng loopback.

8. Trình bày kết quả verify, sao lưu và cách dừng mà vẫn giữ volume.

Giới hạn của bản lab: HTTP localhost với security headers; chưa có HTTPS công khai, thông báo cảnh báo ngoài hệ thống, lịch xoay log file nguồn, high availability hay đánh giá tải lớn. Dữ liệu mẫu gồm sáu công thức. Promtail cần được thay bằng client được hỗ trợ nếu phát triển vận hành lâu dài.

Tài liệu tham khảo chính thức và source

Mã nguồn Recipe
https://github.com/longphan632000/DTC245220017_De29-Recipe

Khung thư mục Blog CMS
https://github.com/Khanhslee/DTC245220024_De7-Blog-CMS

Prometheus tổng quan
https://prometheus.io/docs/introduction/overview/

Hàm PromQL
https://prometheus.io/docs/prometheus/latest/querying/functions/

Grafana datasource Prometheus
https://grafana.com/docs/grafana/latest/datasources/prometheus/

Loki LogQL
https://grafana.com/docs/loki/latest/query/log_queries/

Promtail và vòng đời hỗ trợ
https://grafana.com/docs/loki/latest/send-data/promtail/

Nginx stub status
https://nginx.org/en/docs/http/ngx_http_stub_status_module.html

Phụ lục danh mục hình và nguồn minh chứng

Danh mục giúp đối chiếu số hình trong báo cáo với file ảnh trong repository. UI là ảnh chụp giao diện thật; CLI là hình trích output lệnh có TXT và JSON nguồn. Ảnh sơ đồ thể hiện cấu trúc thiết kế từ cấu hình.

Hình	Nội dung minh chứng	Trang
1	Kiến trúc ứng dụng và các thành phần quan sát	3
2	Phiên bản công cụ được kiểm tra trực tiếp trên Ubuntu	4
3	Repository GitHub có source và các thư mục cấu hình	5
4	Lịch sử bốn mốc commit tại thời điểm chụp minh chứng	5
5	Các commit kỹ thuật và thời điểm từ git log	6
6	Mười một dịch vụ của project Recipe đang chạy	8
7	Các named volume được Docker quản lý	8
8	Các mạng Docker và thành viên tại thời điểm kiểm tra	9
9	Cổng chỉ bind localhost và quyền file bí mật	9
10	Quan hệ giữa năm bảng dữ liệu của ứng dụng Recipe	10
11	Số dòng dữ liệu và quyền DML thực tế của recipe_user	11
12	Năm bảng Recipe và các bảng WordPress cũ được giữ trên máy	11
13	Truy vấn JOIN trả sáu công thức cùng danh mục thời gian khẩu phần	12
14	Trang chủ Recipe được truy cập qua cổng Nginx	13
15	Các thẻ công thức và bộ lọc danh mục	13
16	Từ khóa me chín trả đúng món canh chua	14
17	Danh mục Rau và salad trả một công thức salad	14
18	Thông tin món ăn thời gian và khẩu phần từ database	15
19	Nguyên liệu và hướng dẫn được sắp theo thứ tự	15
20	Biểu mẫu đăng nhập quản trị trước khi nhập mật khẩu	16
21	Trang quản trị hiển thị danh sách công thức hiện có	16
22	Nhập tên mô tả danh mục thời gian và khẩu phần	17
23	Nhập nguyên liệu cách làm và nút lưu công thức	17
24	Thông báo lưu thành công và món mới có tổng thời gian 35 phút	18
25	Biểu mẫu chỉnh sửa cùng bản ghi minh họa	18
26	Tên món được cập nhật và tổng thời gian đổi thành 40 phút	19
27	Bản ghi chuyển sang danh sách đã lưu trữ	19
28	Công thức đã lưu trữ không còn được phục vụ ở trang chi tiết	20
29	Cùng công thức hiển thị lại sau khi khôi phục	20

Hình	Nội dung minh chứng	Trang
30	Kết quả kiểm tra cú pháp cấu hình Nginx	22
31	HTTP 200 403 404 và security headers từ Nginx	23
32	Luồng metrics và log cùng được truy vấn qua Grafana	24
33	Sáu nhóm target Prometheus đều có trạng thái UP	25
34	Endpoint MySQL Exporter và nhãn job instance	25
35	Câu up trả sáu chuỗi dữ liệu có giá trị một	26
36	API Prometheus trả endpoint và health của sáu target	26
37	Hai datasource Loki và Prometheus được khai báo sẵn	27
38	Dashboard hiển thị sáu target UP cùng CPU và RAM của container	28
39	Bốn panel request kết nối MySQL và dữ liệu mạng	30
40	Ba luật cảnh báo đã nạp và đang ở trạng thái INACTIVE	31
41	Explore trả các dòng access log và nhãn nguồn Nginx	33
42	Các request lỗi được lọc bằng trường status sau khi parse JSON	34
43	Các sự kiện đăng nhập thành công có timestamp và user_id	35
44	MySQL error log hiển thị thông báo khởi động và sẵn sàng kết nối	36
45	Các trường UID read only capabilities và privileged từ Docker inspect	37
46	Lệnh id xác nhận UID của app Nginx và MySQL	37
47	MySQL trả mã 1142 khi recipe_user thử tạo bảng	38
48	Nhóm kiểm thử HTTP tìm kiếm CRUD xác thực và database	39
49	Kết quả kiểm thử target metrics datasource dashboard và bốn LogQL	40
50	Kết quả kiểm thử cấu hình runtime và cổng mạng	40
51	Tám kiểm tra cài mới độc lập từ GitHub ở commit đã ghi nhận	41
52	Bản sao lưu mới có quyền 0600 và đọc được định dạng gzip	41

Các file nguồn của hình CLI nằm trong minh-chung/chi-tiet, cùng tên với PNG và phần mở rộng TXT. command-evidence.json giữ command, timestamp, exit code và output. verification.json giữ từng kết quả kiểm thử; reproducibility.json giữ mốc cài mới độc lập. Mật khẩu và bản dump database không nằm trong phần phụ lục công khai.